

自己動手做

Matter over Thread

智慧家庭裝置開發實戰

從安裝 SDK 到做出真正能用的裝置 — 一份給 DIY 玩家的實作指南

nRF54L15 · nRF Connect SDK v3.3.0 · Zephyr · 16 個產品、29 台已實際部署

Ver : 09112026-0.1

這份簡報適合誰

- 你想「做」自己的智慧家庭裝置，而不是「買」現成的
- 你希望裝置能同時原生支援 Apple Home 與 Home Assistant
- 你有一些嵌入式經驗，但 Matter / Thread 對你來說還是全新的
- 你不想再花一年時間，重踩別人踩過的坑

這份簡報和其他教學不一樣的地方

裡面每一件事，都是我們實際做出裝置、裝在兩個家裡跑出來的經驗。

其中「經驗教訓」那一段最有價值 — 每一條都花了我們好幾天才找到。

我們做了什麼

16

個產品

29

台已部署

2

個家

1

套共用程式碼

- 感測器 6 項：門窗、溫濕度 + 氣壓、溫濕度、照度、手勢、空氣品質
- 插座與開關 4 項：智慧插座 (CT)、智慧插座 (HLW8032)、三路開關、攝影機電源開關
- 控制器 5 項：六鍵開關、日立冷氣橋接、電子門鎖、風扇控制、車庫門控制
- 基礎建設 1 項：市電供電的溫濕度 Thread Router，順便擴大網狀網路覆蓋
- $6 + 4 + 5 + 1 = 16$ ，與後面的產品總表逐項對應

第 1 部

基本觀念

Matter 和 Thread 到底是什麼 —
以及它們「不是」什麼。

為什麼選 Matter over Thread ?

- 同一台裝置可同時進 Apple Home、Google Home、Alexa 和 Home Assistant
 - 不需要雲端、不需要廠商帳號、不需要閘道器 — 全部本地溝通
- Thread 是低功耗網狀網路：電池感測器可撐數年，市電裝置負責轉發
 - 具備自我修復能力 — 某個節點掛掉，封包會自動繞路
- 全程 IPv6：每一台裝置都有自己可直接定址的位址
- 開放標準 — 你的裝置不會被單一廠商的 App 綁死

「每一台都可直接定址」到底好在哪？

- 沒有中間人：控制中心直接對裝置講話，不必經過廠商閘道器翻譯
 - 傳統 ZigBee / 433 裝置必須靠一台閘道器代收代發，閘道器壞了全家癱瘓
- 不需要 NAT、不需要 port forwarding — IPv6 位址空間大到每台都分得到
- 可以直接 ping6 到某一台裝置，網路問題能用一般網管工具查
- 群播 (multicast) 可以一次對一群裝置下令，不必逐台重送
- 規模不受閘道器瓶頸限制 — 加裝置就是加節點，不是加閘道器負擔
- 韌體更新、除錯、監控都能直接對裝置做，不必先問廠商雲端

整個技術堆疊，各層在做什麼

你的應用程式

你自己寫的 C/C++ 韌體：讀感測器、驅動繼電器

Matter

資料模型、叢集、配網、安全性

Thread

IPv6 網狀路由、網路建立

IEEE 802.15.4

2.4 GHz 無線電、MAC 層

nRF54L15 硬體

SoC、無線電、周邊

注意：這五層「全部都跑在你的裝置裡」。Apple Home、Google Home、Alexa、Home Assistant 都不在這張圖上 — 它們是另一端的「控制中心」，跑在別台機器上。

裝置角色 — 這個決定會影響一切

角色	供電	無線電	典型用途
FTD (路由節點)	市電	永遠開著	插座、開關、控制器
MTD / SED (睡眠節點)	電池	會睡眠	門窗、溫濕度、照度感測器
ICD (LIT / SIT)	電池	輪詢父節點	Matter 針對睡眠裝置的規範
Border Router (邊界路由器)	市電	永遠開著	HomePod、Apple TV、HA + 網卡

為什麼重要

睡眠裝置只有在「輪詢」的那一瞬間才聽得到封包。所有的逾時、重試、恢復行為，都和市電裝置完全不同。我們最難搞的 bug 幾乎都出在這裡。

Fabric 與配網 (Commissioning)

- FABRIC 就是「某一個控制中心的網路」— Apple Home 是一個，HA 是另一個
- 一台裝置可以同時加入多個 Fabric (multi-admin)，我們的裝置兩邊都加
- COMMISSIONING (配網) 就是把裝置加進某個 Fabric 的那段交握流程
 - 先用 BLE 完成交握，裝置再加入 Thread 網路
 - 控制中心會先驗證裝置的憑證，通過才信任它
- 連線工作階段 (CASE Session) 存在 RAM 裡 — 一斷電就消失，必須重建

CASE Session — 平常在跑的其實是它

Matter 裡有兩種安全通道，用途完全不同：

	PASE	CASE
全名	Passcode Authenticated	Certificate Authenticated
何時使用	只在「配網」那一次	配網之後的每一次通訊
靠什麼認證	8 位數 passcode (SPAKE2+)	雙方的營運憑證 NOC
承載	BLE	Thread / IPv6

- CASE 用 Sigma1 / Sigma2 / Sigma3 三段交握，雙方互相驗證憑證後產生對稱金鑰
- 金鑰只存在 RAM — 斷電、重開機、韌體更新之後全部消失
- 重開機後裝置必須「重新建立」每一條 CASE 通道，才會再開始送資料
 - 這就是為什麼斷電後感測器要一段時間才回得來 (見教訓 2)
- CASE Resumption 可以用上次的憑證略過完整交握，比較省電也比較快

第 2 部

硬體

我們用的晶片與模組 —
以及為什麼選它們。

nRF54L15 — 主晶片

項目	規格	為什麼重要
CPU	Arm Cortex-M33 128 MHz	跑 Matter + Thread 綽綽有餘
Flash (RRAM)	1524 KB	Matter 很肥，空間一定要夠
RAM	256 KB	Thread、Matter、BLE 要同時常駐
無線電	2.4 GHz 多協定	BLE 與 802.15.4 共用同一顆無線電
安全性	TrustZone、KMU	存放 DAC 私鑰

「多協定共用無線電」是個很微妙的重點：BLE 和 Thread 是輪流用的。BLE 事件太長會餓死 Thread — 我們的配網不只一次因此失敗。

E73-2G4M08S1F 模組 — 為什麼選它

- 真正的好處是「高度整合、體積很小」：
 - nRF54L15 + 32 MHz HFXO + 32.768 kHz LFXO 全部包在同一個屏蔽罩裡
 - DCDC 電感、天線與匹配電路也都做在模組內
 - 等於少畫一整塊 RF 電路，載板只要處理電源與 I/O
- RF 佈線最難的部分（阻抗控制、匹配、天線淨空區）都不用自己處理

不要誤會：法規認證不是好處

這顆模組有中國的認證，但「沒有台灣 NCC 認證」。所以在台灣，模組的既有認證幫不上忙，要上市一樣得自己送驗。選它的理由是整合度與體積，不是省認證。

共用的是「慣例」，不是同一片板子

- 16 個產品「不是」共用同一片 PCB — 每個產品有自己的板子
- 真正共用的是「同一組 GPIO 慣例」，用在每個產品都有的共通功能：
 - 狀態指示 LED → P1.10
 - 回復原廠設定按鍵 → P1.06
 - 框架要求的第二顆 LED → P1.14
- 好處：同一套 Board 程式碼、同一套除錯流程，可以套用在任何一個產品上

只有「溫濕度、氣壓、照度、手勢」這四類真的共用一片板子 — 因為它們全都是 I2C 裝置，接線完全一樣，差別只在焊哪一顆感測器。

載板共用腳位 — 每個產品都一樣

這三支腳定義在載板的 .dts 裡，不在各產品的 overlay：

腳位	功能	設定
P1.06	回復原廠按鍵 (sw0)	PULL_UP ACTIVE_LOW，長按 ≥ 6 秒清除 Fabric
P1.10	狀態 LED (led0)	ACTIVE_HIGH，只在配網與長按時亮
P1.14	保留 LED (led1)	ACTIVE_HIGH，未接線 — 純粹滿足框架要求

P1.14 那顆「假 LED」是因為 Nordic 的 Matter Board 函式庫會無條件初始化第二顆 LED。車庫門控制器需要 P1.14 當門磁輸入，所以它把這顆改接到 P0.04。

CR2032 能撐多久？怎麼算、怎麼報

- CR2032 標稱 220 mAh，扣掉脈衝負載與 2.0 V 截止，可用約 200 mAh
- 六個電池產品全部「每 300 秒醒一次」，刻意和 ICD 慢輪詢對齊
 - 一次醒來就做完：讀感測器、讀電池、輪詢父節點，再一起睡回去
- 算下來「睡眠電流」壓倒性地決定壽命，取樣本身幾乎不影響

電壓	回報電量	說明
≥ 2900 mV	100%	平台區，輕載下電壓幾乎不動
2700 mV	70%	拐點 — 開始明顯掉電
2400 mV	20%	已經在陡降段，該換電池了
≤ 2000 mV	0%	截止電壓

刻意用「分段」而不是線性換算：線性會嚴重高估剩餘壽命 — 顯示 30% 時，真實的 CR2032 其實只剩約 5%，使用者幾乎沒有預警時間。

電量回報的兩個平台差異

- HomeKit 一定要有 BatPercentRemaining 才會顯示電量
 - 只給 BatChargeLevel (OK / Warning / Critical) 的話，HomeKit 會整個忽略
 - Home Assistant 則只要 BatChargeLevel 就滿足了
- Power Source cluster 必須放在 EP0，EndpointList = {0}
 - 放在 EP1 在 HA 看得到，但在 HomeKit 完全看不到
- 門檻要加遲滯（我們用 30 mV），否則在邊界會一直跳動

注意：本簡報所有續航數字都是「依資料手冊與設定值計算」出來的，不是實測值。CR2032 內阻高，射頻脈衝會造成壓降，實際壽命通常明顯短於理論值。

nRF54L15 的 SAADC 腳位對照 (陷阱)

類比輸入編號和 port 編號「不是」連號的，只有 port 1 能當類比輸入：

```
AIN0 = P1.04    AIN1 = P1.05    AIN2 = P1.06    AIN3 = P1.07  
AIN4 = P1.11    AIN5 = P1.12    AIN6 = P1.13    AIN7 = P1.14
```

我們真的踩進去過

智慧插座的電流互感器接在 P1.04，但 overlay 寫成 AIN4。AIN4 其實是 P1.11 — 一顆被內部提升電阻拉到 3.3 V 的按鍵。ADC 因此穩定讀到 3789 codes，換算出「永遠不變的 12–14 W」，看起來完全正常。

電池裝置則完全不用外部分壓：直接讀 NRF_SAADC_VDD 這個內部電源電壓即可。

第 3 部



開發環境

安裝 nRF Connect SDK 與 VS Code —
多數人卡關就卡在這裡。

nRF Connect SDK (NCS) 到底是什麼？

- NCS 不是「一個東西」，它是一整包：
 - Zephyr RTOS — 核心、驅動、devicetree、Kconfig
 - Nordic 的驅動與函式庫 — 無線電、MPSL、電源管理
 - Matter (connectedhomeip) — 整套 Matter 協定堆疊，以模組形式納入
 - 一組鎖定版本的工具鏈 — 編譯器、CMake、Python、west
- 「west」是把這些 repository 串起來的統合工具

版本一定要鎖死。我們全線統一用 v3.3.0。Matter 和 Zephyr 都在快速演進，混版本是最快讓你損失一個週末的方法。

VS Code 環境建置 — 一步一步來

1

安裝 Visual Studio Code

code.visualstudio.com

2

安裝 nRF Connect for VS Code Extension Pack

從 Marketplace 安裝

3

打開 nRF Connect 側邊欄 → Manage toolchains

會裝好編譯器與相關工具

4

用同一個面板安裝 SDK v3.3.0

會下載 Zephyr、Matter，全部一次到位

5

安裝 SEGGER J-Link

用來燒錄與抓 RTT log

6

安裝 nRF Util (nrfutil)

命令列燒錄工具

驗證安裝是否成功

打開 nRF Connect 的終端機（它會自動幫你設好 PATH），然後檢查：

```
west --version
nrfutil --version
nrfutil device list          # 應該要看到你的 J-Link
arm-zephyr-eabi-gcc --version
```

如果 nrfutil device list 顯示「Supported devices found: 0」，代表 J-Link 根本沒被列舉出來 — 先把 USB 線拔掉重插，再去查其他問題。這一點我們重複浪費過很多次時間。

第 4 部

專案結構

每個檔案分別在做什麼。

資料夾結構

```
apps/my_product/  
  CMakeLists.txt      要編譯哪些原始碼  
  prj.conf            Kconfig 選項 – 最主要的設定檔  
  Kconfig             這個產品專屬的 Kconfig 預設值  
  sysbuild.conf       bootloader / 多映像檔建置設定  
  pm_static_<board>.yaml Flash 分割區配置  
  VERSION             韌體版本 ( DFU 會用到 )  
  boards/<board>.overlay devicetree : 你的感測器、腳位、匯流排  
src/  
  main.cpp            進入點  
  app_task.cpp        Matter 初始化 + 你的應用邏輯  
  zcl_callbacks.cpp   外部送進來的 Matter 指令會落在這裡  
  default_zap/        Matter 資料模型 ( 見第 6 部 )  
prj_production_overlay.conf 出貨版建置設定
```

prj.conf — 最主要的設定檔

CONFIG_CHIP=y	啟用 Matter
CONFIG_CHIP_DEVICE_VENDOR_ID=65524	0xFFF4 (測試用 VID)
CONFIG_CHIP_DEVICE_PRODUCT_ID=32769	0x8001
CONFIG_CHIP_DEVICE_DISCRIMINATOR=0x080	BLE 配對識別碼
CONFIG_CHIP_DEVICE_SPAKE2_PASSCODE=<你的 8 位數配對碼>	
CONFIG_CHIP_DEVICE_TYPE=21	0x0015 門窗感測器

Kconfig 是階層式的，裡面有一堆你「沒設定過」的預設值。當行為怪怪的時候，去讀「產生出來的」設定檔 `build/<app>/zephyr/.config`，而不是 `prj.conf`。兩者並不一樣，而且產生出來的那個才是真的。

Devicetree overlay — 描述你的硬體

```
&i2c30 {  
    status = "okay";  
    clock-frequency = <I2C_BITRATE_STANDARD>;  
  
    sht35: sht3xd@44 {  
        compatible = "sensirion,sht3xd";  
        reg = <0x44>;  
    };  
};
```

- Devicetree 描述「硬體長什麼樣」；Kconfig 決定「要不要編進對應的軟體」
- 兩邊都要做：這裡加節點，prj.conf 裡也要有 CONFIG_SHT3XD=y

第 5 部

建置與燒錄

從原始碼到一台可以配對的裝置。

建置 (Build)

```
west build -b e73_battery_sensor/nrf54l15/cpuapp \  
  -d C:/nrf-builds/my_product \  
  -p always \  
  C:/nrf-projects/apps/my_product \  
  -- -DBOARD_ROOT=C:/nrf-projects
```

- -p always = 完全重建。比較慢，但可以避免「舊設定殘留」這種鬼故事
- -d 把建置輸出放在原始碼樹「外面」—— 保持 repo 乾淨
- BOARD_ROOT 告訴 west 你自訂的開發板定義放在哪
- 我們把這串包成 build.cmd，這樣沒人需要背它

燒錄 (Flash)

```
nrftutil device program --firmware merged.hex \  
    --options chip_erase_mode=ERASE_ALL --x-family nrf54l  
nrftutil device reset --x-family nrf54l
```

抹除模式	會保留什麼	什麼時候用
ERASE_ALL	什麼都不留	更換身分識別、要一個乾淨的起點
(預設)	保留 settings_storage	純更新韌體，保留已配對狀態

ERASE_ALL 會把 Matter 的 Fabric 一起清掉 — 裝置必須在每一個生態系裡重新配對一次。如果你只是要更新韌體，就不要加這個參數。

Debug 版 vs. 出貨版

設定	Debug 版	出貨版
Log / RTT	開啟	關閉
LTO 最佳化	關閉	開啟 (映像檔小約 20%)
RESET_ON_FATAL_ERROR	n — 當機後永遠停住	y — 自動重開
Shell	有	移除

這裡有個陷阱

Debug 版遇到嚴重錯誤時是「停住」而不是重開機。所以一台當掉的裝置，外觀上和一台卡死的裝置一模一樣 — 換成出貨版，它其實早就重開了。

第 6 部

Matter 資料模型

Endpoint、Cluster，以及 ZAP 工具。

Endpoint 與 Cluster

- ENDPOINT (端點) = 一個邏輯裝置。EP0 永遠是 root (裝置資訊、網路設定)
- CLUSTER (叢集) = 一項功能。OnOff、BooleanState、TemperatureMeasurement...
- ATTRIBUTE (屬性) = 叢集裡的一個值。控制中心讀取與訂閱的就是它
- DEVICE TYPE (裝置類型) = 告訴控制中心「這個端點是什麼」(0x0015 = 門窗感測器)

EP0	root (根端點)	Basic Information, Network Commissioning
EP1	溫度感測器	TemperatureMeasurement
EP2	濕度感測器	RelativeHumidityMeasurement
EP3	氣壓感測器	PressureMeasurement

ZAP — 以及一個讓每個人損失一天的陷阱

- ZAP 是用來定義 Endpoint 與 Cluster 的圖形 / JSON 工具
- 它會把結果產生成 C++ 程式碼，放進 src/default_zap/zap-generated/

BYPASS_IDL

建置流程「不會」從你的 .zap 檔重新產生 zap-generated/。也就是說，你改了 .zap 卻沒手動跑產生器，等於什麼都沒改。我們就這樣出過一版錯誤的資料模型。

所以：每次改完 .zap，一定要手動跑一次重新產生的腳本。

第 7 部

憑證與 Factory Data

沒有人會跟你解釋的那一段 —
也是你的裝置拒絕配對的原因。

憑證鏈 (Certificate Chain)

PAA	Product Attestation Authority — 最上層的根憑證
PAI	Product Attestation Intermediate — 中間層憑證
DAC	Device Attestation Certificate — 每個產品一份，內含 VID + PID

Home Assistant 的 Matter Server 只信任它認得的 PAA。我們最後把每一個產品都改成掛在 SDK 標準測試 PAA 底下 — 自簽的 PAA 會直接被擋掉，訊息是「PAA not found in trust store」，裝置根本配不進去。

Certification Declaration (CD)

- 一段簽章過的資料，用來宣告「這組 VID + PID 是通過認證的真實產品」
- 它裡面帶著一份「涵蓋哪些 Product ID」的清單
- 在 Nordic 平台上，它是以巨集的形式編在 App 映像檔裡 — 不在 factory data

這件事害我們整整除錯了一整段時間

我們把 CD 擴大到涵蓋更多 PID，然後又開了一個新產品 — 但那個新產品的資料夾裡，還放著「舊版」的 CD，涵蓋範圍不含它的 PID。結果就是完全無法配網。每個產品都各自帶一份 CD，而複製出來的檔案一定會走鐘。

配對碼是由三樣東西組成的

- DISCRIMINATOR (12 bits) — BLE 廣播時用來分辨「是哪一台」
- PASSCODE (8 位數) — SPAKE2+ 的共享密鑰，真正的安全來源
- QR / 手動配對碼 — 把上面兩個加上 VID / PID 編碼起來
- 三者都改了就要重印標籤 — QR 是「當下這組識別」的快照

關鍵觀念

Discriminator 只是「認人」，Passcode 才是「鎖」。所以 Discriminator 必須唯一，但不必保密；Passcode 可以共用，但必須當成密碼看待。

規則一：Discriminator 用「配」的，不是「算」的

```
discriminator = (product_code << 7) | unit_seq // 共 12 bits  
                5 bits (1-31)         7 bits (0-127)
```

- 為什麼不像序號那樣用 SoC ID 去雜湊？因為它只有 12 bits = 4096 個值
 - 同一個產品 30 台，碰撞機率約 10%
 - 一個家裡 80 台裝置，碰撞機率高達約 78% — 完全不能用
- 結構化配號則是「結構上不可能碰撞」，不是靠機率
- 序號用推導、Discriminator 用配號 — 需求不同，就該用不同機制

規則二：為什麼這裡可以接受比較弱的保證

	序號重複時	Discriminator 過期時
後果	HA 出現幽靈裝置	QR 掃不出對應的裝置
何時發現	可能永遠不會發現	配網當下立刻發現
修復方式	要回頭清資料庫	重印一張標籤
性質	沉默的失敗	會自己喊出來的失敗

會自己喊出來的失敗，容忍度可以比較高。真正需要「絕對不能重複」的，是那些出錯了你也不會知道的東西。

規則三：unit_seq 永遠不用手打

- provision.py 先讀板子的 SoC ID，再決定要用哪個號碼：
 - 已登記過的板子 → 沿用 units.csv 裡「原本就記著」的 discriminator
 - 全新的板子 → 自動配下一個還沒用過的 unit_seq
- 已記錄的值一律「照抄」，絕不重算
 - 所以重新燒錄永遠不會偷偷改掉 discriminator，標籤上的 QR 不會失效
- 接錯板子時它會直接擋下來，不會把 A 產品的韌體燒進 B 產品

```
python provision.py --product camera_switch  
# units.csv: unit_seq, serial, discriminator, flashed, location, notes
```

規則四：QR 是「每台一張」，手動配對碼不是

- 11 位數的手動配對碼只帶 discriminator 的 bits 11:8
 - 在這套配法下，那幾個位元等於 $\text{product_code} \gg 1$
 - 也就是說：它和 `unit_seq` 無關 — 同產品的每一台都一樣
- 於是同時得到兩種能力：
 - 每台專屬的 QR → 精準指定要配哪一台
 - 全產品共用的手動碼 → 標籤掉了、看不清時的後備方案

2026-08-28 在 `camera_switch` 上實測：三台產生三組完全不同的 QR，但手動配對碼一模一樣 — 和推論完全一致。

規則五：Passcode 刻意全產品共用

- 整個產品線共用同一組 8 位數 passcode，這是刻意的決定
 - 之所以安全，是因為 discriminator 本來就唯一 — 不會配錯台
 - 好處：不用管理 16 組密碼，也不會有「這台到底是哪一組」的問題
- Matter 規範禁止一些過於簡單的值，不要用：
 - 00000000、11111111 ... 99999999 (八位同一個數字)
 - 12345678、87654321

但是請注意

共用 passcode 只適合「全部都是自己家、自己管」的情況。只要裝置會交到別人手上，就要改成每台一組；而且 passcode 永遠不要推上公開的 repo。

配對碼的三個實戰教訓

改 PID 會讓舊的 QR 直接失效

QR 裡編了 VID / PID / discriminator / passcode。只要動到其中任何一個，標籤就得重印 — 我們換 PID 時就必須重新產生 factory_data.png。

真的發生過 discriminator 撞號

temphumidity-router 原本是 0xE93，和 Door_Controller 完全一樣。兩台廣播同一個 discriminator，配網工具根本分不出是哪一台。

複製來的 gen_qr.py 印出別的產品的號碼

3-cut-switch 的 gen_qr.py 編碼的是 Car_Door 的 discriminator — 又是一次「複製貼上沒改乾淨」。

Discriminator、Passcode、QR Code

項目	作用	唯一性要求
Discriminator	BLE 配對階段用來辨識裝置	附近範圍內必須唯一
Passcode	8 位數配對密碼	可以共用
QR Code	把 VID/PID/Discriminator/Passcode 編碼進去	印在標籤上
序號 (Serial)	App 上顯示；HA 用它當裝置主鍵	務必唯一

Home Assistant 的裝置註冊表「只用序號」當主鍵。兩台裝置如果序號相同，在 HA 裡會被合併成「同一台裝置」，兩邊的實體全部擠在一起。

第 8 部

規模化的裝置身分管理

這套架構讓我們不再做出
「重複身分」的裝置。

問題出在哪

- 序號原本是編譯時期的固定常數 — 每一台燒出來的序號都一樣
- Home Assistant 把它們合併了：一台裝置底下掛著來自 3 個產品的 35 個實體
- 更糟的是，好幾個「不同產品」因為複製貼上而共用同一組常數
 - 六鍵開關和日立冷氣橋接器，兩個都回報 DZNRF54L00103
 - 手勢感測器和空氣品質感測器，兩個都回報 DZNRF54L00105

解法 — 讓身分從晶片本身推導出來

```
DZ54L-01-B7196C
|      |  |
|      |  +-- SoC 唯一 ID 的 FNV-1a 雜湊 ( 24 bits )
|      +----- 產品代碼
+----- 廠商 + 平台
```

- 同一份韌體映像檔可以燒進每一台裝置 — 每台自己算出自己的身分
- 重新燒錄會算出「一模一樣」的序號，所以 HA 裡的裝置條目不會跑掉
- 不需要計數器、不需要每台專屬的 factory data、也沒有會弄丟的配號表

一個產品代碼，決定所有東西

```
產品代碼 -> PID          = 0x8000 + code  
          -> DAC 前綴      = pNN-  
          -> Discriminator = (code << 7) | unit    ( 每產品 127 台 )  
          -> 序號前綴      = DZ54L-NN-
```

- 產品代碼是「永久」的，絕不重複使用 — 即使該產品已經停產
- 如此一來，衝突是「結構上不可能發生」，而不是靠人自律
- 配號腳本會讀晶片 ID、配一個台號、記錄到 CSV 裡

第 9 部

產品清單

16 個產品，一套共用程式碼。

產品總表 (1 / 2)

產品	代碼	PID	裝置類型	供電	台數
contact_sensor (門窗感測器)	1	0x8001	0x0015	電池	4
TempHumidity_BMP280 (溫濕壓)	2	0x8002	0x0302	電池	1
Lux_sensor (照度感測器)	3	0x8003	0x0106	電池	1
SixButton (六鍵開關)	4	0x8004	0x000F	電池	1
HitachiAC (日立冷氣橋接)	5	0x8005	0x0301	市電	3
Gesture_sensor (手勢感測器)	6	0x8006	0x000F	電池	1
Smart_Plug (智慧插座)	8	0x8008	0x010A	市電	2
3-cut-switch (三路開關)	9	0x8009	0x010A	市電	2

產品總表 (2 / 2)

產品	代碼	PID	裝置類型	供電	台數
camera_switch (攝影機開關)	10	0x800A	0x010A	市電	4
ENV_sensor (空氣品質)	11	0x800B	0x002C	市電	2
Pulse_Lock (電子門鎖)	12	0x800C	0x000A	市電	0
fan_control (風扇控制)	15	0x800F	0x002B	市電	2
Door_Controller (車庫門)	17	0x8011	0x0202	市電	1
temphumidity-router (路由)	18	0x8012	0x0302	市電	1
TempHumidity_SHT35 (溫濕度)	19	0x8013	0x0302	電池	1
AC_SmartPlug (插座 HLW8032)	20	0x8014	0x010A	市電	1

接下來每一個產品各有一頁規格：用途、硬體接線、Matter 資料模型、韌體模組，以及該產品上實際遇到的狀況。

門窗感測器 · Contact Sensor

apps/contact_sensor/

磁鐵靠近 / 離開時回報門窗開關狀態，並回報電池電量。

產品代碼	Product ID	Matter 裝置類型	端點	供電 / Thread 角色	Bootloader / OTA	已部署
01	0x8001	0x0015 Contact Sensor	2	電池 / ICD 睡眠裝置	MCUboot + BLE DFU	4 台

硬體重點	Matter 資料模型	韌體模組
· 霍爾感測器 → P0.01 · 無 I2C — 全專案中最單純的感測硬體 · SAADC 直接量測內部 VDD 當電池電量	· EP1 BooleanState (StateValue) · EP1 PowerSource (電池電量 / 狀態) · ICD：慢輪詢 300 秒、閒置 300 秒 · ICD：活躍門檻 15 秒、快輪詢 500 ms	· battery_monitor.cpp、watchdog.cpp

CR2032 續航估算	可用電量 ~200 mAh	喚醒週期 300 秒	平均電流 ~5.2 μA	理論壽命 ~4.3 年	實務預期 1.5–2.5 年 （事件驅動：霍爾中斷；每 300 秒取樣電池）
----------------	------------------	---------------	-----------------	----------------	---

GPIO 一覽 (含板上共用的 LED 與按鍵)

P0.01 gpio0 SAADC	霍爾感測器輸入 (ACTIVE_LOW) sense-edge-mask 0x2：P0.01 邊緣喚醒 電池電壓 (內部 VDD，無外部分壓)	P1.06 P1.10 P1.14	回復原廠按鍵 (長按 ≥ 6 秒) 狀態 LED (配網時亮) 保留 LED (框架需要，未接線)
-------------------------	---	-------------------------	---

實戰筆記

教訓 1 (bt_rx 堆疊溢位) 與教訓 2 (六小時退避) 都是在這個產品上找出來的。

門窗感測器 · Contact Sensor

電路圖

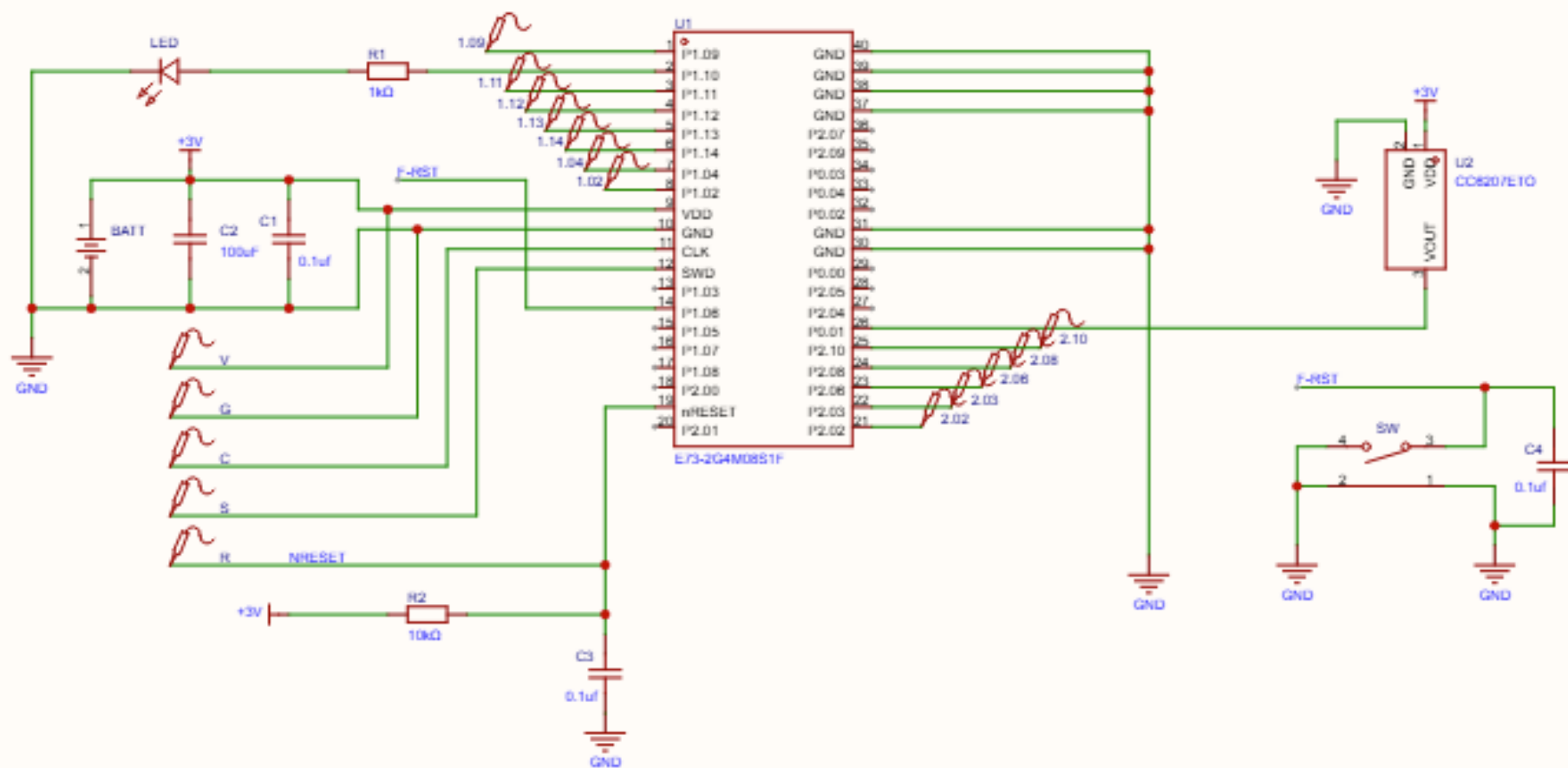
GPIO

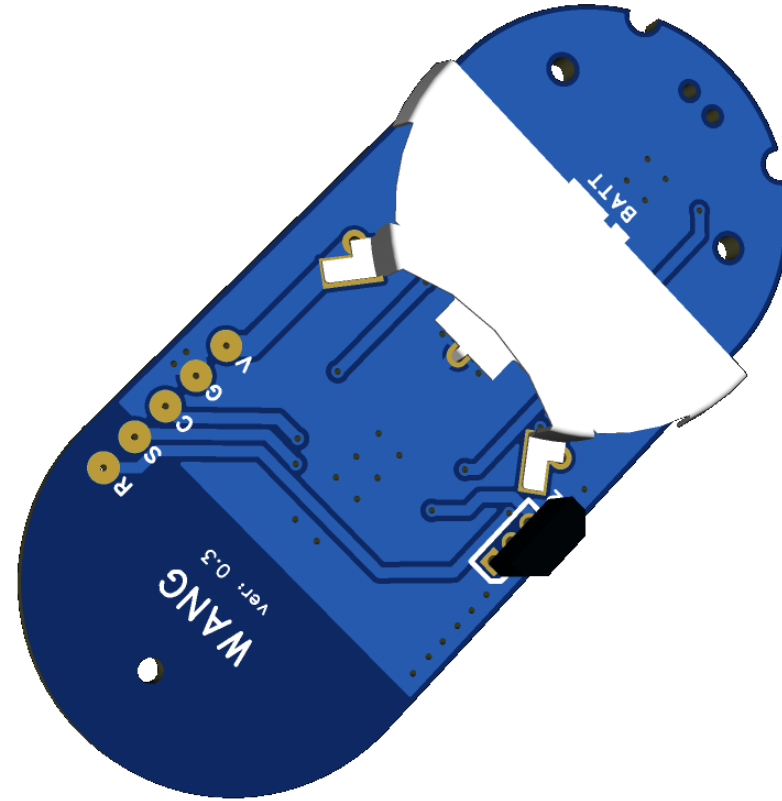
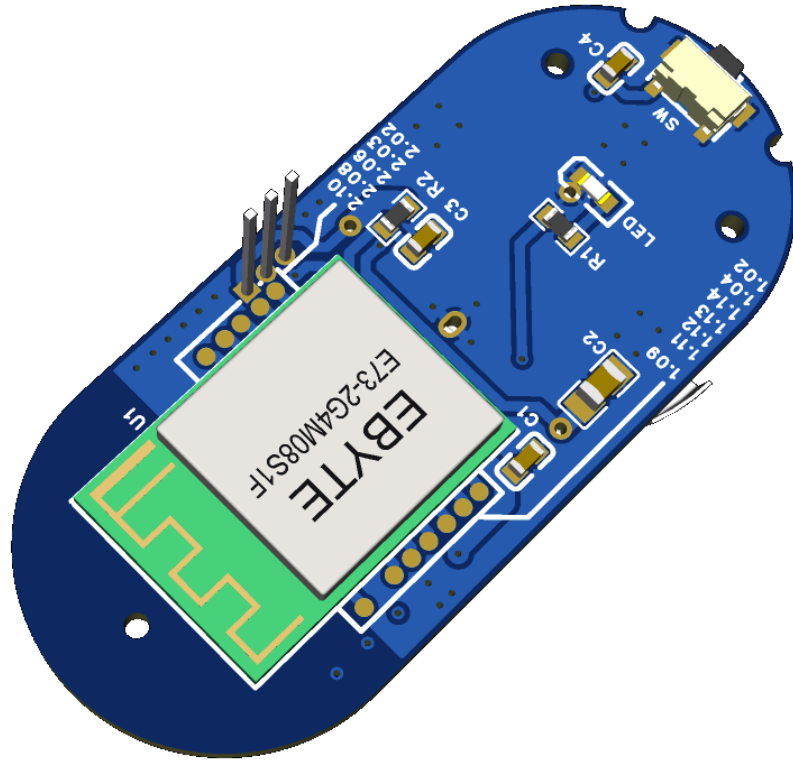
P1.10 : LED

P1.14: Fake LED to meet the requirement of Nordic Matter Board library.

P0.01 :

P1.06 : Factory Reset





溫濕度 + 氣壓感測器 · Temp / Humidity / Pressure Sensor

apps/TempHumidity_Sensor_BMP280/

同一片板子上放兩顆感測器，一次回報溫度、濕度與大氣壓力。

產品代碼	Product ID	Matter 裝置類型	端點	供電 / Thread 角色	Bootloader / OTA	已部署
02	0x8002	0x0302 Temperature Sensor	4	電池 / ICD 睡眠裝置	無 (v1 無 bootloader)	1 台

硬體重點	Matter 資料模型	韌體模組
· SHT35 @ 0x44 (溫度 + 濕度) · BMP280 @ 0x76 (驅動用 bosch,bme280) · 兩顆共用 i2c30 · SCL P0.00 / SDA P0.01 · SAADC 內部 VDD 量測電池	· EP1 TemperatureMeasurement · EP2 RelativeHumidityMeasurement · EP3 PressureMeasurement · EP1 PowerSource (電池)	· temp_humidity_sensor.cpp · pressure_sensor.cpp · battery_monitor.cpp · watchdog.cpp

CR2032 續航估算	可用電量 ~200 mAh	喚醒週期 300 秒	平均電流 ~5.4 µA	理論壽命 ~4.2 年	實務預期 1.5–2 年 (每 300 秒讀 SHT35 + BMP280 · 各約 10–15 ms)
----------------	------------------	---------------	-----------------	----------------	---

GPIO 一覽 (含板上共用的 LED 與按鍵)					
P0.00	I2C30 SCL (TWIM30)	SAADC	電池電壓 (內部 VDD · 無外部分壓)		
P0.01	I2C30 SDA (TWIM30)	P1.06	回復原廠按鍵 (長按 ≥ 6 秒)		
I2C 0x44	SHT35 溫濕度感測器	P1.10	狀態 LED (配網時亮)		
I2C 0x76	BMP280 氣壓感測器	P1.14	保留 LED (框架需要 · 未接線)		

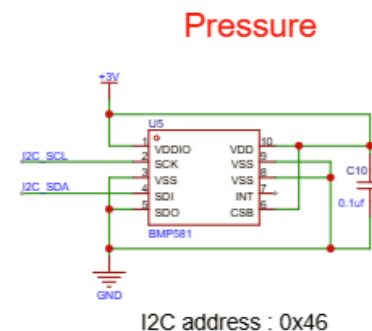
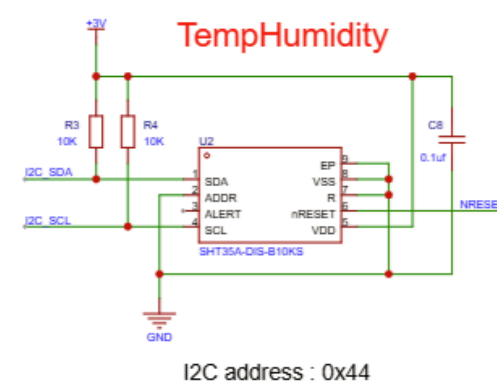
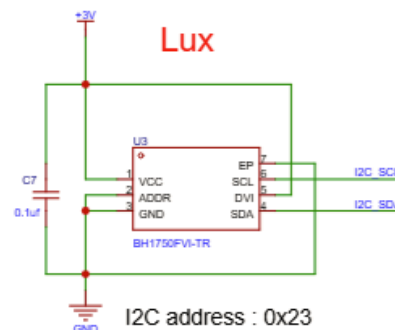
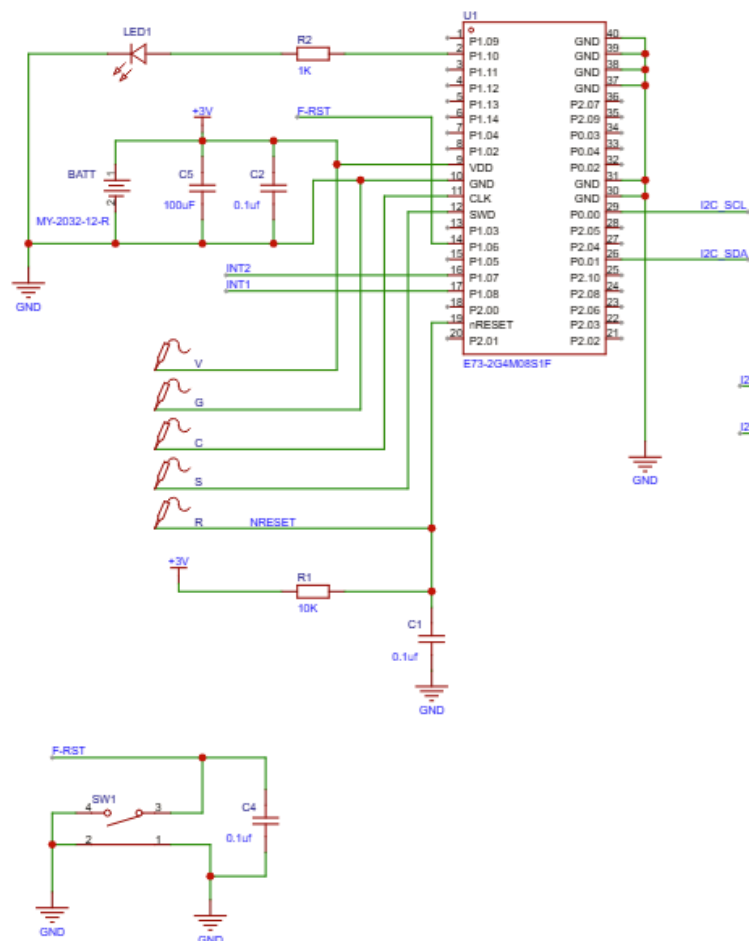
實戰筆記

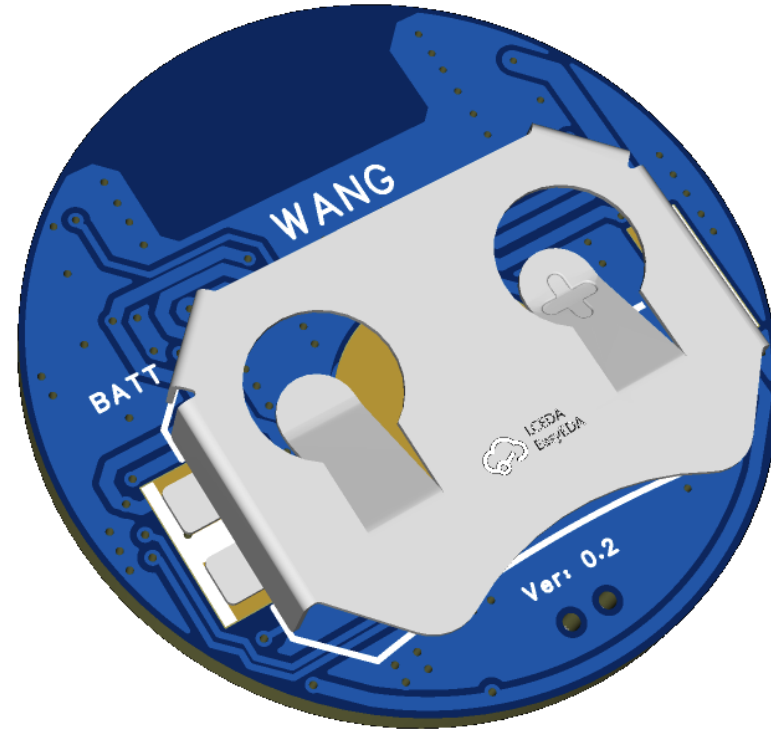
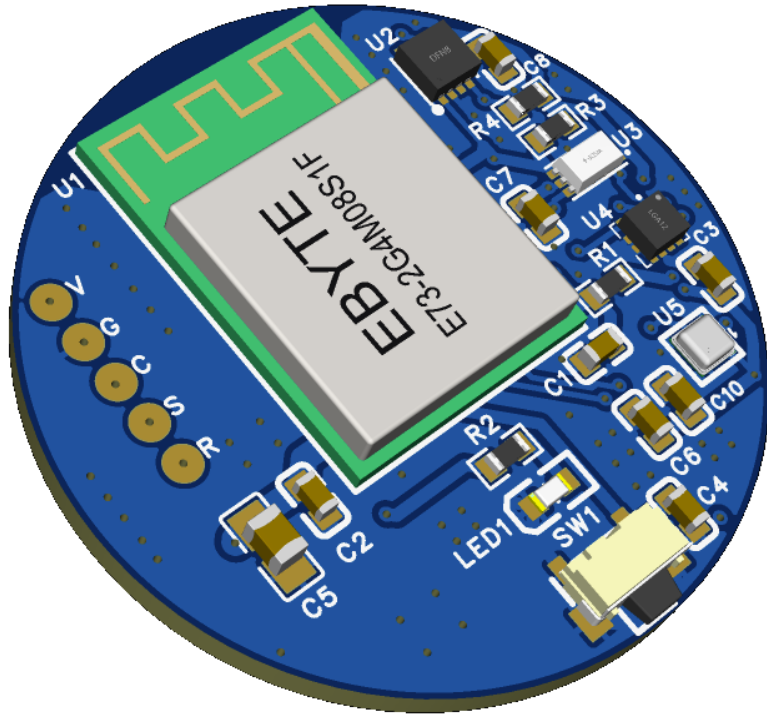
教訓 7 的來源：市面 BMP280 山寨品的溫度通道永遠回傳 0，導致氣壓補償算錯。

溫濕度 + 氣壓感測器 · Temp / Humidity / Pressure Sensor

電路圖

GPIO
P1.10 : LED
P1.14 : Fake LED to meet Nordic Matter Board library.
P0.00 : I2C_SCL
P0.01 : I2C_SDA
P1.06 : Factory Reset
P1.07 : INT2
P1.08 : INT1





照度感測器 · Lux Sensor

apps/Lux_sensor/

量測環境照度，用來做「天黑自動開燈」這類自動化的觸發條件。

產品代碼	Product ID	Matter 裝置類型	端點	供電 / Thread 角色	Bootloader / OTA	已部署
03	0x8003	0x0106 Light Sensor	2	電池 / ICD 睡眠裝置	無 (v1 無 bootloader)	1 台

硬體重點	Matter 資料模型	韌體模組
· BH1750 @ 0x23 (數位照度計) · i2c30 : SCL P0.00 / SDA P0.01 · SAADC 內部 VDD 量測電池	· EP1 IlluminanceMeasurement · EP1 PowerSource (電池) · ICD 參數與門窗感測器相同	· lux_sensor.cpp、battery_monitor.cpp

CR2032 續航估算	可用電量 ~200 mAh	喚醒週期 300 秒	平均電流 ~5.5 µA	理論壽命 ~4.1 年	實務預期 1.5–2 年 (每 300 秒 BH1750 單次量測 · H 模式約 180 ms)
----------------	------------------	---------------	-----------------	----------------	--

GPIO 一覽 (含板上共用的 LED 與按鍵)

P0.00	I2C30 SCL (TWIM30)	P1.06	回復原廠按鍵 (長按 ≥ 6 秒)
P0.01	I2C30 SDA (TWIM30)	P1.10	狀態 LED (配網時亮)
I2C 0x23	BH1750 數位照度計	P1.14	保留 LED (框架需要 · 未接線)
SAADC	電池電壓 (內部 VDD · 無外部分壓)		

實戰筆記

Apple Home 的配件磁貼清單不含 Light Sensor (0x0106) · iOS 上要另外處理顯示方式。

Refer to Page 56 and 57

六鍵無線開關 · Six-Button Switch

apps/SixButton/

六個獨立按鍵，每一顆在 Matter 裡都是一個獨立的開關端點，可各自綁定不同場景。

產品代碼	Product ID	Matter 裝置類型	端點	供電 / Thread 角色	Bootloader / OTA	已部署
04	0x8004	0x000F Generic Switch	7	電池 / ICD 睡眠裝置	無 (v1 無 bootloader)	1 台

硬體重點	Matter 資料模型	韌體模組
6 個按鍵全在 gpio1 (有 GPIOTE · 可觸發中斷) - 全部 GPIO_PULL_UP GPIO_ACTIVE_LOW - 回復原廠鍵 P1.06 是為了這個產品的走線而選的 - SAADC 內部 VDD 量測電池	- EP1–EP6 各一個 Switch cluster - 共 7 個端點 (含 EP0 root) - EP1 PowerSource (電池)	six_button.cpp、battery_monitor.cpp

CR2032 續航估算	可用電量 ~200 mAh	喚醒週期 300 秒	平均電流 ~5.2 µA	理論壽命 ~4.3 年	實務預期 1.5–2.5 年 (事件驅動：6 顆按鍵中斷；每 300 秒取樣電池)
----------------	------------------	---------------	-----------------	----------------	--

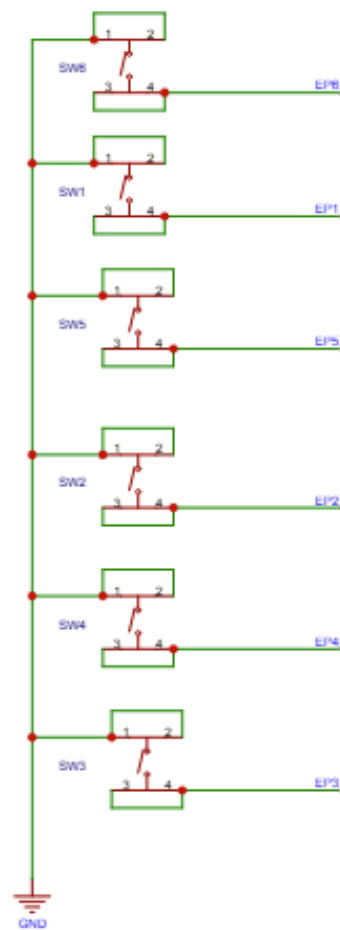
GPIO 一覽 (含板上共用的 LED 與按鍵)					
P1.05	按鍵 1 → EP1	P1.08	按鍵 5 → EP5	P1.06	回復原廠按鍵 (長按 ≥ 6 秒)
P1.04	按鍵 2 → EP2	P1.07	按鍵 6 → EP6	P1.10	狀態 LED (配網時亮)
P1.09	按鍵 3 → EP3	全部	PULL_UP ACTIVE_LOW	P1.14	保留 LED (框架需要 · 未接線)
P1.11	按鍵 4 → EP4	SAADC	電池電壓 (內部 VDD · 無外部分壓)		

實戰筆記

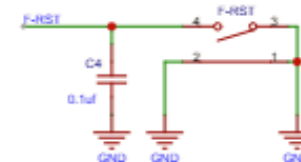
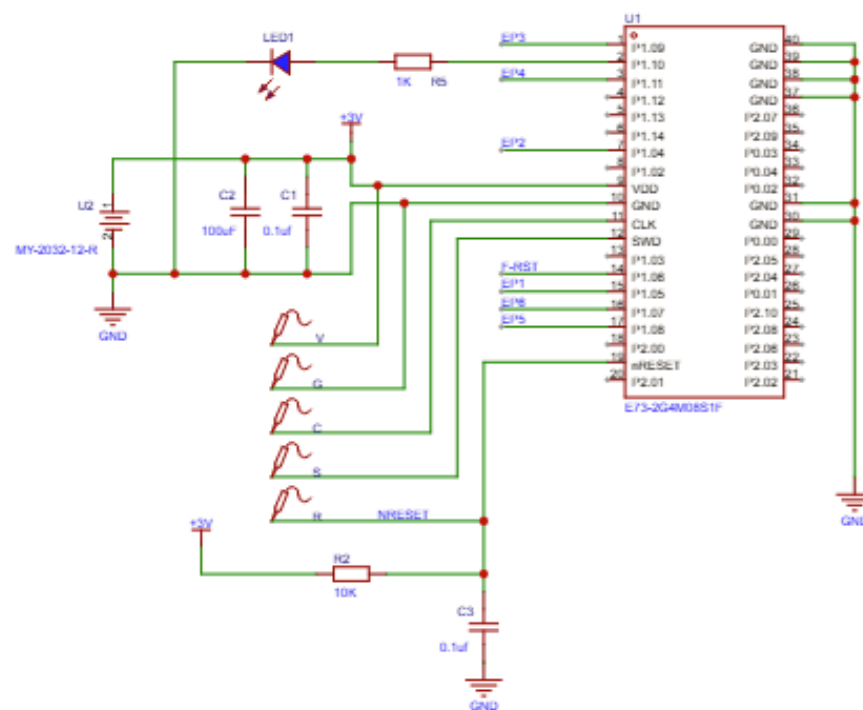
曾與日立冷氣橋接器共用同一組寫死序號 (DZNRF54L00103) · 是教訓 5 的案例之一。

六鍵無線開關 · Six-Button Switch

電路圖

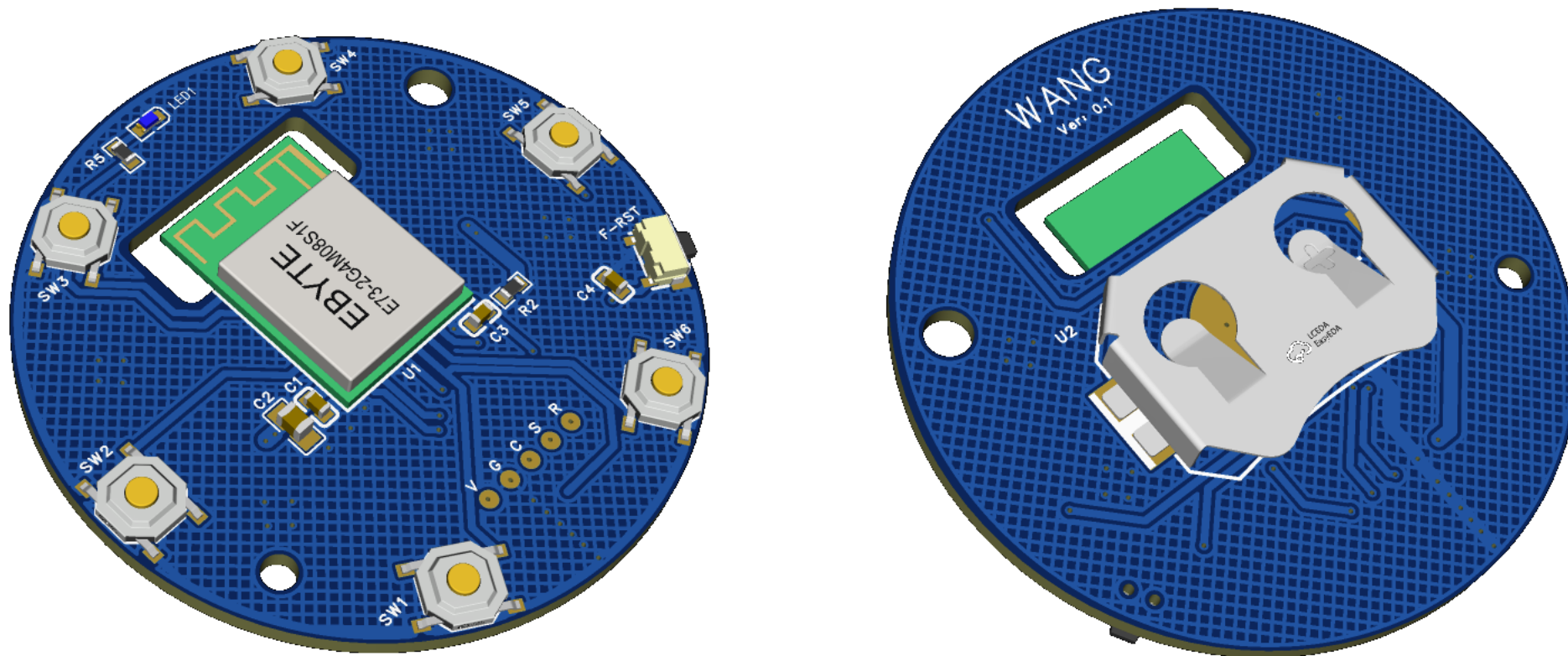


GPIO
P1.10: LED
P1.14: Fake LED
P1.05: EP1
P1.04: EP2
P1.09: EP3
P1.11: EP4
P1.08: EP5
P1.07: EP6
P1.06: Factory Reset



六鍵無線開關 · Six-Button Switch

PCBA-3D



日立冷氣橋接器 · Hitachi A/C Bridge

apps/HitachiAC/

透過冷氣機原廠的 TaiSEIA (泰協) 串列介面控制日立冷氣，讓它變成標準的 Matter 恆溫器。

產品代碼	Product ID	Matter 裝置類型	端點	供電 / Thread 角色	Bootloader / OTA	已部署
05	0x8005	0x0301 Thermostat	2	市電 / Thread 路由節點	MCUboot + BLE DFU	3 台

硬體重點	Matter 資料模型	韌體模組
· UART20 接冷氣機的 TaiSEIA 介面 · RX P1.07 / TX P1.08 — 都在 gpio1 電源域 · SERIAL20 原生驅動 port 1，所以 UART 走這裡 · 無額外感測器 — 室溫由冷氣機本身回報	· EP1 Thermostat (模式、設定溫度、室溫) · EP1 FanControl (風速) · 以 Thermostat 出現在 Apple Home 與 HA	· taixia_uart.cpp (協定收發) · taixia_climate.cpp (狀態機) · temperature_manager.cpp

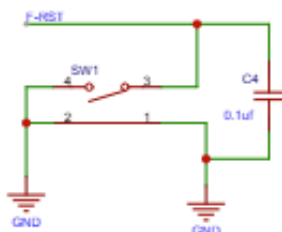
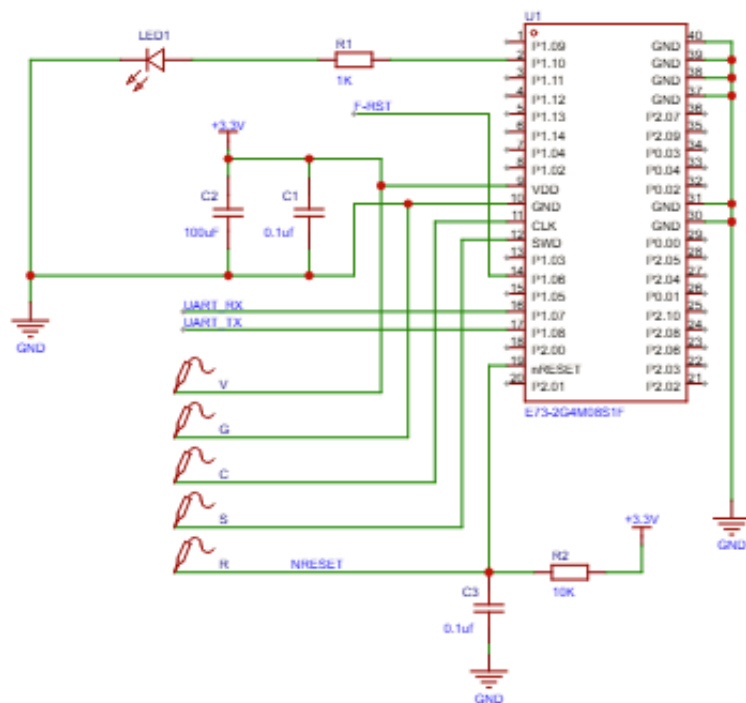
GPIO 一覽 (含板上共用的 LED 與按鍵)			
P1.07	UART20 RX ← 冷氣機	P1.10	狀態 LED (配網時亮)
P1.08	UART20 TX → 冷氣機	P1.14	保留 LED (框架需要，未接線)
P1.06	回復原廠按鍵 (長按 ≥ 6 秒)		

實戰筆記

兩台舊機的 bootloader 仍是舊金鑰，必須用 SWD 燒錄；OTA 會「成功但不生效」(教訓 3)。

日立冷氣橋接器 · Hitachi A/C Bridge

電路圖



GPIO

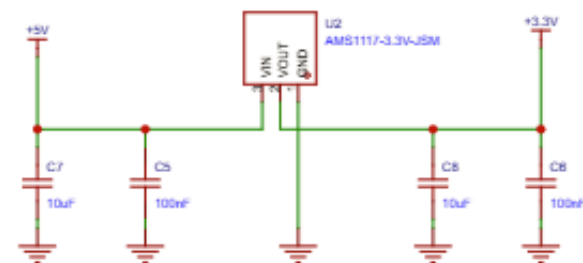
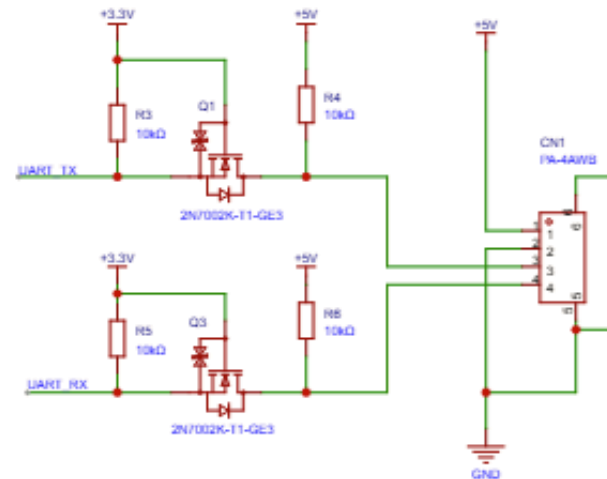
P1.10 : LED

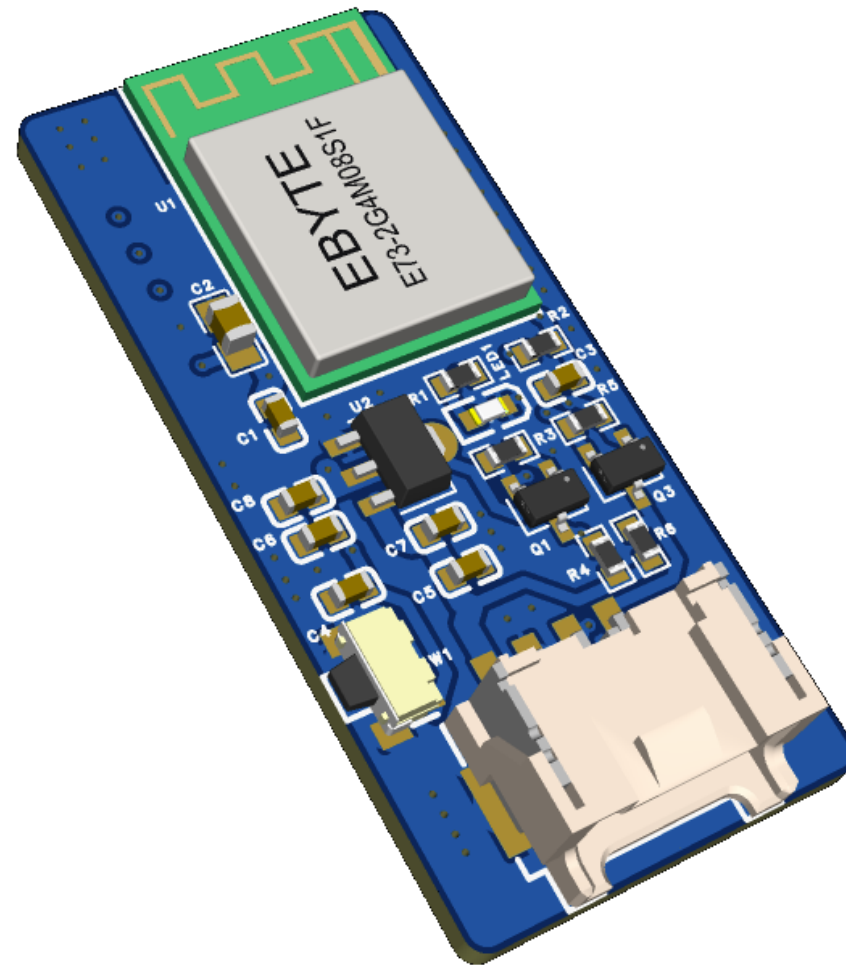
P1.14 : Fake LED to meet Nordic Matter Board library.

P1.08 : UART_TX

P1.07 : UART_RX

P1.06 : Factory Reset





手勢感測器 · Gesture Sensor

apps/Gesture_sensor/

用加速度計偵測敲擊與翻轉等動作，把「手勢」轉成 Matter 的開關事件。

產品代碼	Product ID	Matter 裝置類型	端點	供電 / Thread 角色	Bootloader / OTA	已部署
06	0x8006	0x000F Generic Switch	2	電池 / ICD 睡眠裝置	無 (v1 無 bootloader)	1 台

硬體重點	Matter 資料模型	韌體模組
· LIS2DW12 @ 0x18 (三軸加速度計) · 中斷腳 → P1.08 · 平時可完全睡著 · i2c30 : SCL P0.00 / SDA P0.01 · SAADC 內部 VDD 量測電池	· EP1 Switch (手勢對應成按鍵事件) · EP1 PowerSource (電池)	· gesture.cpp (中斷驅動的手勢判定) · battery_monitor.cpp

CR2032 續航估算	可用電量 ~200 mAh	喚醒週期 300 秒	平均電流 ~6.7 µA	理論壽命 ~3.4 年	實務預期 1–2 年 (LIS2DW12 低功耗常開偵測約 1.5 µA · 是最大額外負擔)
----------------	------------------	---------------	-----------------	----------------	--

GPIO 一覽 (含板上共用的 LED 與按鍵)

P0.00	I2C30 SCL (TWIM30)	SAADC	電池電壓 (內部 VDD · 無外部分壓)
P0.01	I2C30 SDA (TWIM30)	P1.06	回復原廠按鍵 (長按 ≥ 6 秒)
P1.08	LIS2DW12 中斷 IRQ (ACTIVE_HIGH)	P1.10	狀態 LED (配網時亮)
I2C 0x18	LIS2DW12 三軸加速度計	P1.14	保留 LED (框架需要 · 未接線)

實戰筆記

中斷驅動，平時完全睡著 — 是所有電池裝置裡最省電的一種設計。

Refer to Page 56 and 57

智慧插座 · Smart Plug

apps/Smart_Plug/

遠端 / 本地開關電源，並用電流互感器量測負載功率。

產品代碼	Product ID	Matter 裝置類型	端點	供電 / Thread 角色	Bootloader / OTA	已部署
08	0x8008	0x010A On/Off Plug-in Unit	2	市電 / Thread 路由節點	MCUboot + BLE DFU	2 台

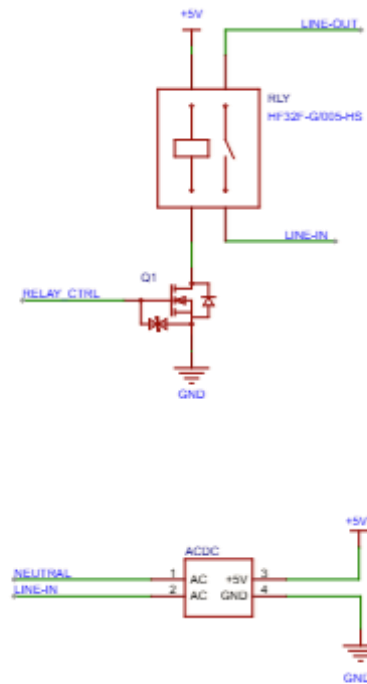
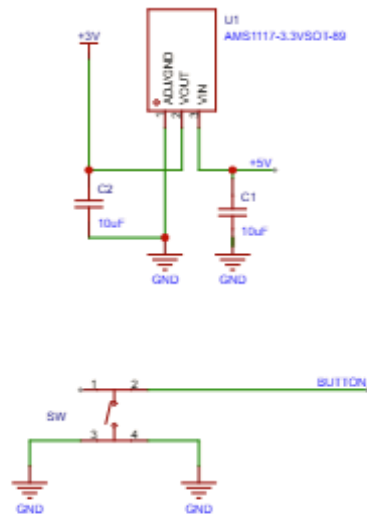
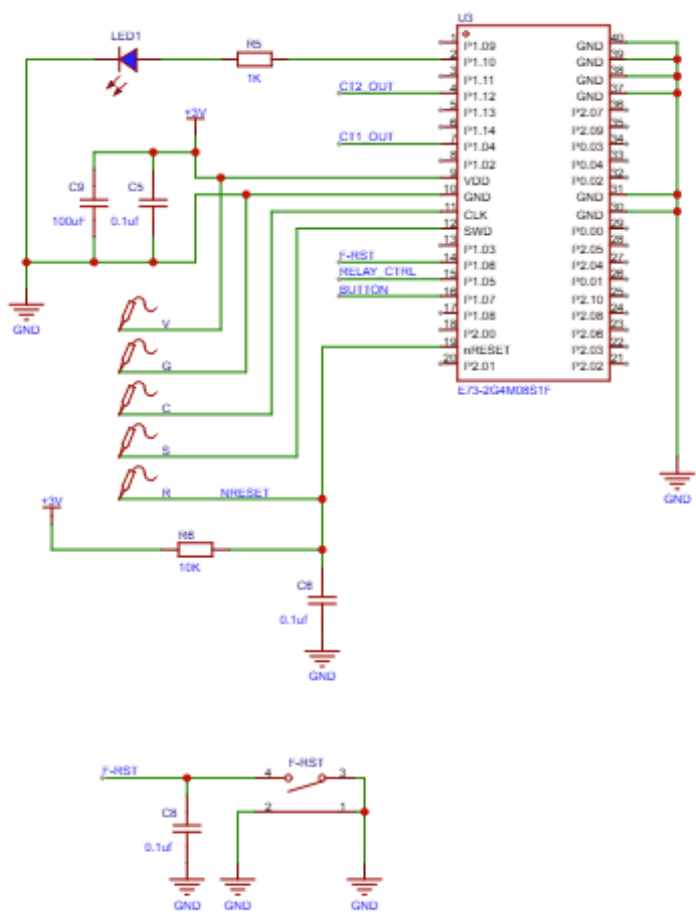
硬體重點	Matter 資料模型	韌體模組
- 繼電器 → P1.05、實體按鍵 → P1.07 - CT → P1.04 = SAADC AIN0 (不是 AIN4 !) - CT 輸出騎在 R4/R8 10k/10k 分壓的 ~1.5 V 偏壓上 - P1.05 與 P1.07 都在 gpio1 · 有 GPIOTE 可中斷	- EP1 OnOff - EP1 ElectricalPowerMeasurement (功率 / 電流) - EP1 Identify	- relay_control.cpp、metering.cpp - ElectricalPowerMeasurementDelegate.cpp - ble_config_service.cpp、watchdog.cpp

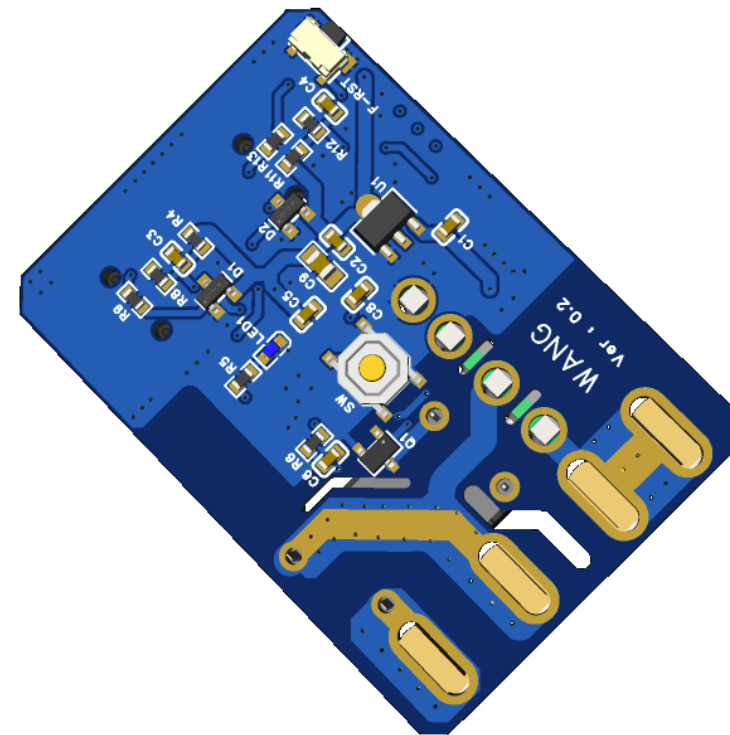
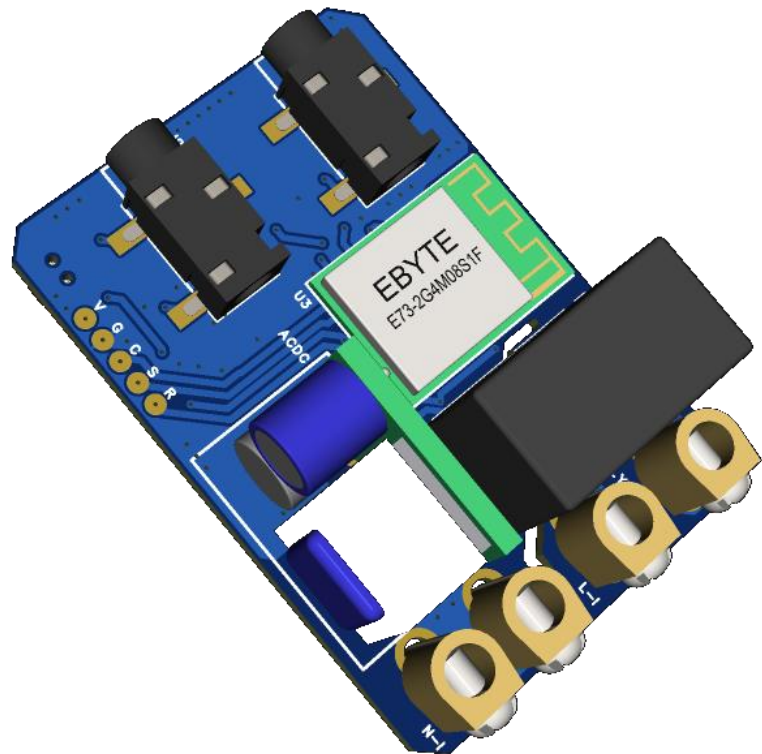
GPIO 一覽 (含板上共用的 LED 與按鍵)			
P1.05	繼電器 (ACTIVE_HIGH)	P1.06	回復原廠按鍵 (長按 ≥ 6 秒)
P1.07	實體按鍵 (PULL_UP / ACTIVE_LOW)	P1.10	狀態 LED (配網時亮)
P1.04	電流互感器 CT → SAADC AIN0	P1.14	保留 LED (框架需要 · 未接線)
ADC	12-bit · gain 1/4 · 扣除 1.5 V 直流偏壓		

實戰筆記

AINx 曾誤設為 AIN4 (其實是 P1.11) · ADC 一直在量按鍵的提升電阻，讀出固定不變的 12–14 W。

GPIO
P1.10: LED
P1.14: Fake LED
P1.05: RELAY_CTRL
P1.04: CT1_OUT
P1.12: CT2_OUT
P1.11: OUT1
P1.07: BUTTON
P1.06: Factory Reset





三路電燈開關 · Three-Gang Switch

apps/3-cut-switch/

取代傳統三聯電燈開關：三路各自可獨立控制，實體按鍵與 App 兩邊都能操作。

產品代碼	Product ID	Matter 裝置類型	端點	供電 / Thread 角色	Bootloader / OTA	已部署
09	0x8009	0x010A On/Off Plug-in Unit	4	市電 / Thread 路由節點	MCUboot + BLE DFU	2 台

硬體重點	Matter 資料模型	韌體模組
- 繼電器 1/2/3 → P1.05 / P1.07 / P1.08 - 實體按鍵 1/2/3 → P1.11 / P1.12 / P1.13 (PULL_UP) - CT → P1.04 · 量測切換後的總負載電流 - 硬體佈局承襲自 Car_Door 專案	- EP1 / EP2 / EP3 各一個 OnOff - 共 4 個端點 (含 EP0 root) - ElectricalPowerMeasurement (總功率)	- switch_control.cpp、metering.cpp - ElectricalPowerMeasurementDelegate.cpp

GPIO 一覽 (含板上共用的 LED 與按鍵)					
P1.05	繼電器 1 → EP1	P1.12	實體按鍵 SW-2 → EP2	P1.10	狀態 LED (配網時亮)
P1.07	繼電器 2 → EP2	P1.13	實體按鍵 SW-3 → EP3	P1.14	保留 LED (框架需要 · 未接線)
P1.08	繼電器 3 → EP3	P1.04	電流互感器 CT → SAADC		
P1.11	實體按鍵 SW-1 → EP1	P1.06	回復原廠按鍵 (長按 ≥ 6 秒)		

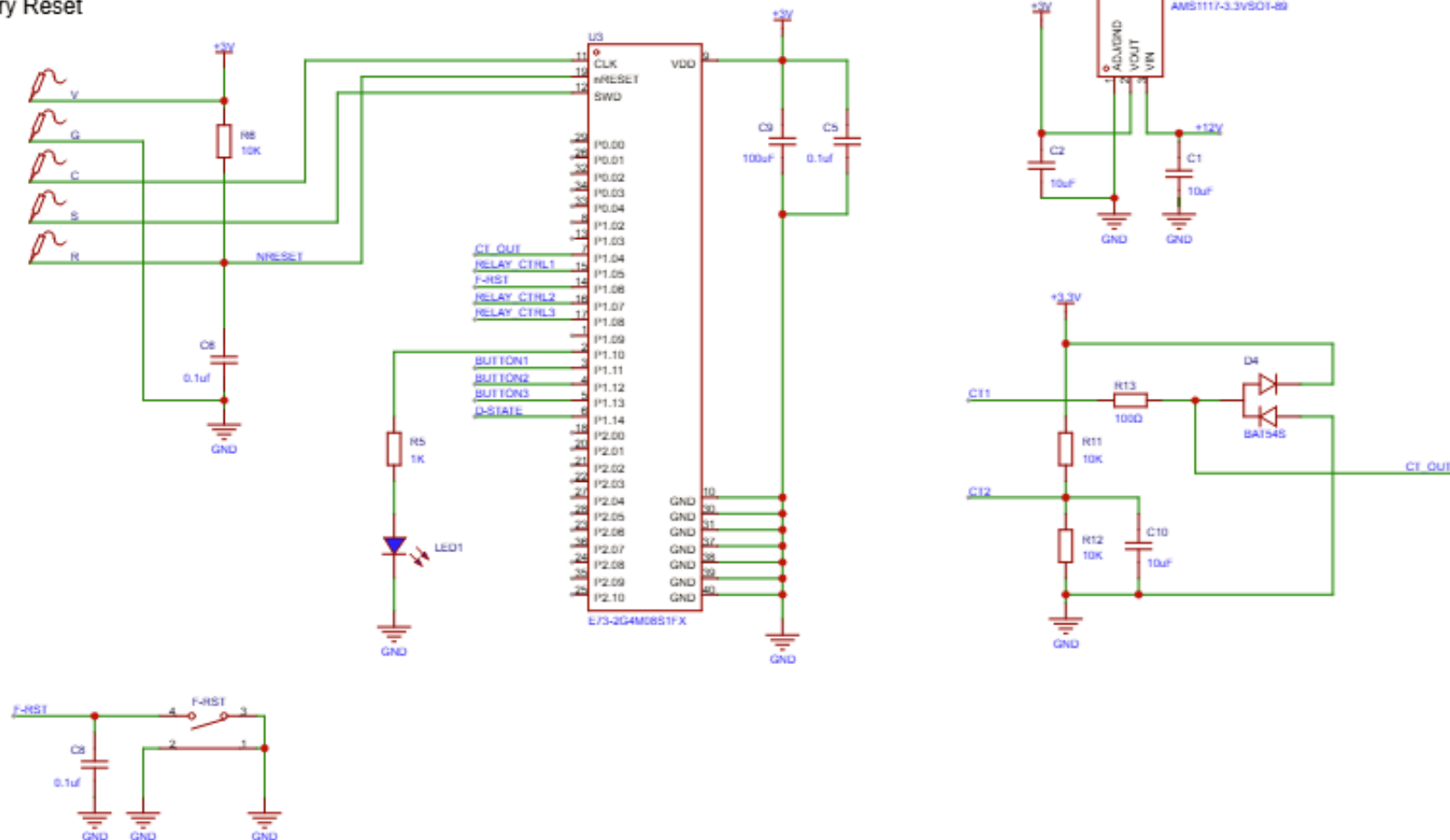
實戰筆記

△ overlay 仍宣告 CT 為 NRF_SAADC_AIN4，但 AIN4 是 P1.11 (SW-1 按鍵)。Smart_Plug 與 Door_Controller 已改為 AIN0，這一個尚未修正。

三路電燈開關 · Three-Gang Switch

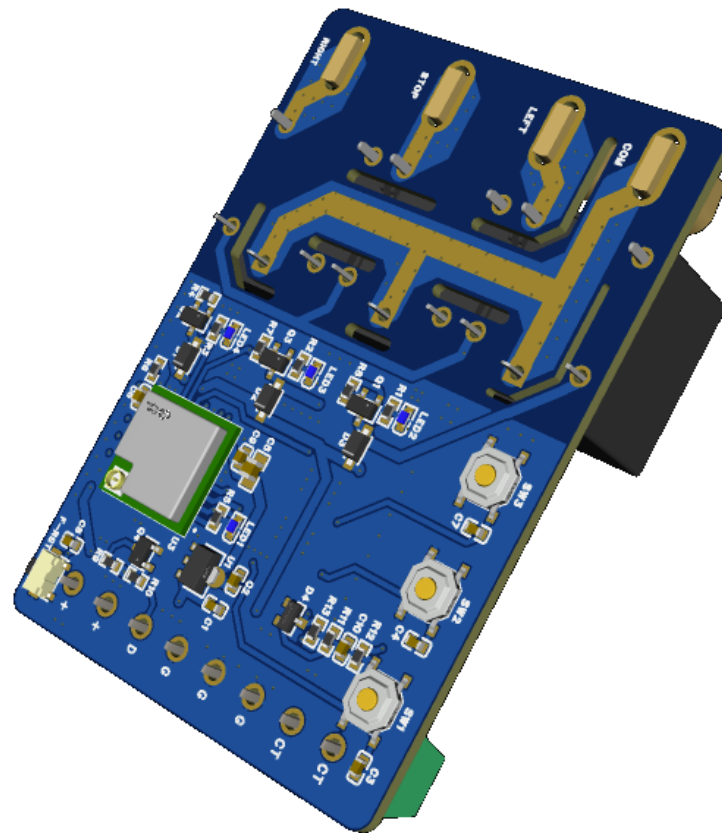
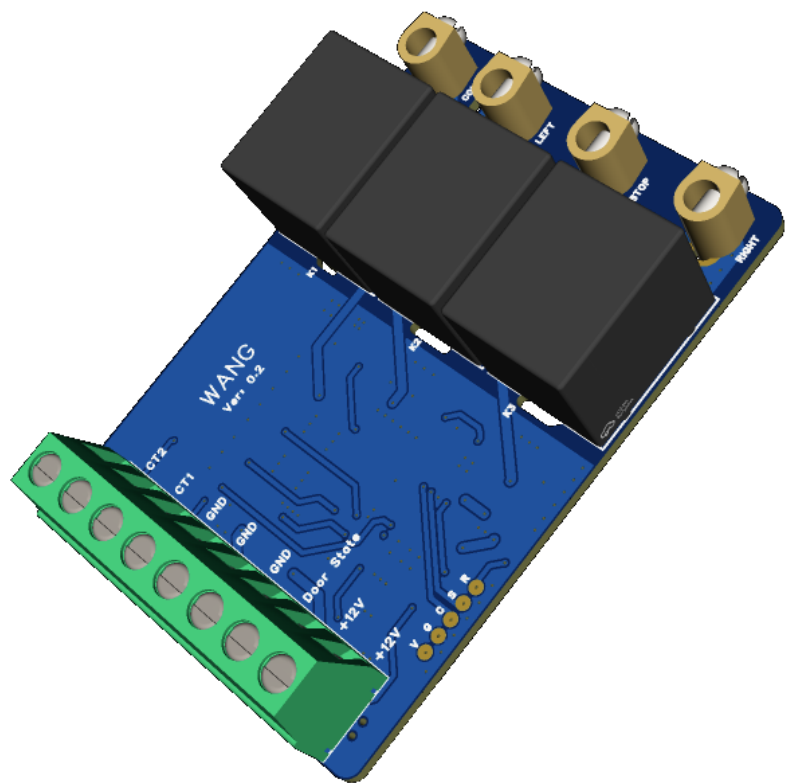
電路圖

GPIO
P1.04 : CT output; 20/1V; 1.65V bias
P1.10: LED
P1.14: Fake LED
P1.05: RELAY_CTRL1
P1.07: RELAY_CTRL2
P1.08: RELAY_CTRL3
P1.11: BUTTON1
P1.12: BUTTON2
P1.13: BUTTON3
P1.06: Factory Reset



三路電燈開關 · Three-Gang Switch

PCBA-3D



攝影機電源開關 · Camera Power Switch

apps/camera_switch/

單純把攝影機的電源切開 / 接上，用來做「在家時關閉監視器」這類隱私自動化。

產品代碼	Product ID	Matter 裝置類型	端點	供電 / Thread 角色	Bootloader / OTA	已部署
10	0x800A	0x010A On/Off Plug-in Unit	2	市電 / Thread 路由節點	MCUboot + BLE DFU	4 台

硬體重點	Matter 資料模型	韌體模組
· 單一繼電器 → P0.01 · 無感測器、無 CT、無 I2C、無 ADC · 除了板上共用的 LED 與按鍵之外，只多一支腳 · 整個產品線中硬體最精簡的一個	· EP1 OnOff · EP1 Identify · 共 2 個端點	· relay_control.cpp、ble_config_service.cpp、watchdog.cpp

GPIO 一覽 (含板上共用的 LED 與按鍵)			
P0.01	繼電器 (ACTIVE_HIGH)	P1.10	狀態 LED (配網時亮)
P1.06	回復原廠按鍵 (長按 ≥ 6 秒)	P1.14	保留 LED (框架需要，未接線)

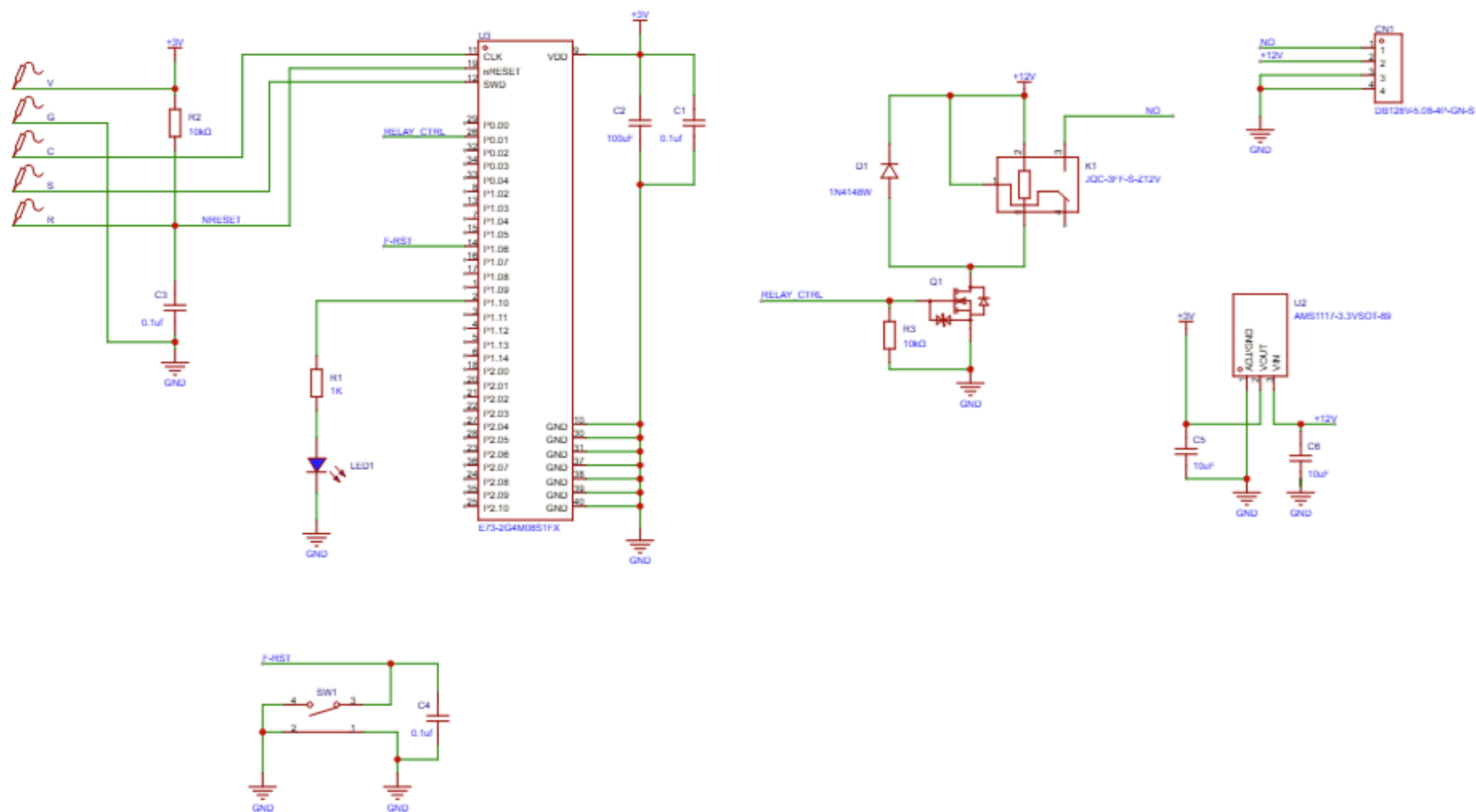
實戰筆記

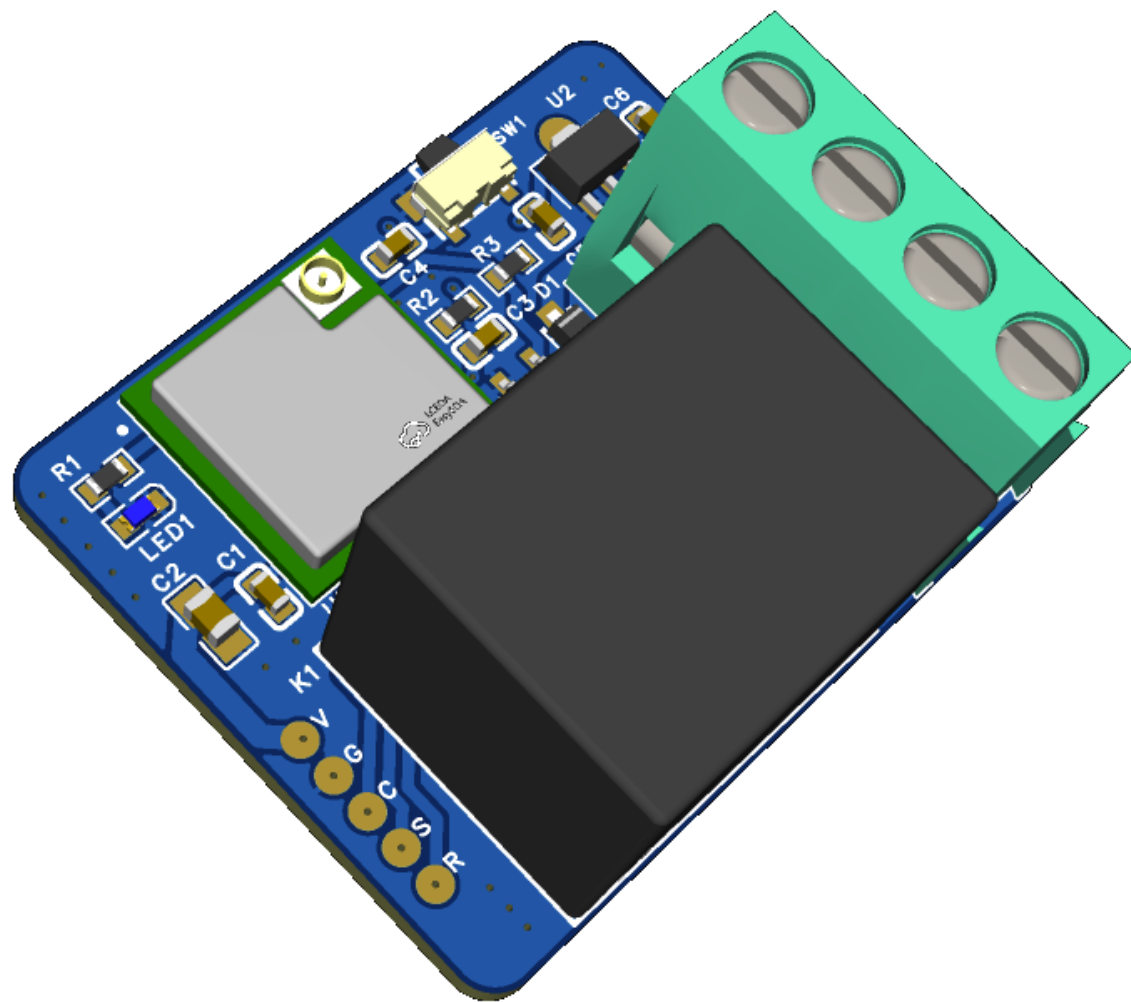
如果你要挑一個產品當作「第一個自己做的 Matter 裝置」，就從這個開始。

攝影機電源開關 · Camera Power Switch

電路圖 1 / 2

GPIO
P1.10 : LED
P1.14: Fake LED to meet the requirement
of Nordic Matter Board library.
P0.01 : Relay control
P1.06 : Factory Reset





空氣品質感測器 · Air Quality Sensor

apps/ENV_sensor/

量測二氧化碳濃度、溫度、濕度與照度，是整個產品線中端點最多的感測器。

產品代碼	Product ID	Matter 裝置類型	端點	供電 / Thread 角色	Bootloader / OTA	已部署
11	0x800B	0x002C Air Quality Sensor	5	市電 / Thread 路由節點	MCUboot + BLE DFU	2 台

硬體重點	Matter 資料模型	韌體模組
· SCD41 @ 0x62 (CO2 + 溫度 + 濕度) · BH1750 @ 0x23 (照度) · 兩顆共用 i2c30 : SCL P0.00 / SDA P0.01 · 市電供電 — SCD41 耗電高，不適合電池	· EP1 AirQuality / CarbonDioxideConcentration · EP2 TemperatureMeasurement · EP3 RelativeHumidityMeasurement · EP4 IlluminanceMeasurement	· scd41_sensor.cpp、lux_sensor.cpp · dfu_advertising.cpp

GPIO 一覽 (含板上共用的 LED 與按鍵)					
P0.00	I2C30 SCL (TWIM30)	—	P1.06	市電供電，無電池量測 ADC	
P0.01	I2C30 SDA (TWIM30)		P1.10	回復原廠按鍵 (長按 ≥ 6 秒)	
I2C 0x62	SCD41 : CO2 + 溫度 + 濕度		P1.14	狀態 LED (配網時亮)	
I2C 0x23	BH1750 照度計			保留 LED (框架需要，未接線)	

實戰筆記

曾與手勢感測器共用同一組寫死序號 (DZNRF54L00105)，在 HA 裡被合併成一台裝置。

空氣品質感測器 · Air Quality Sensor

電路

GPIO

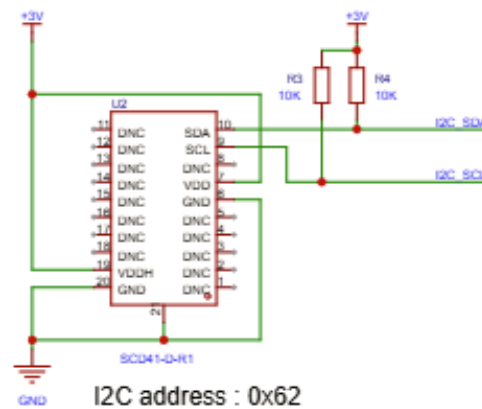
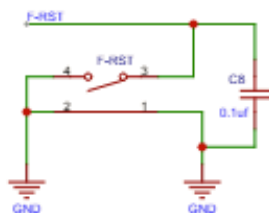
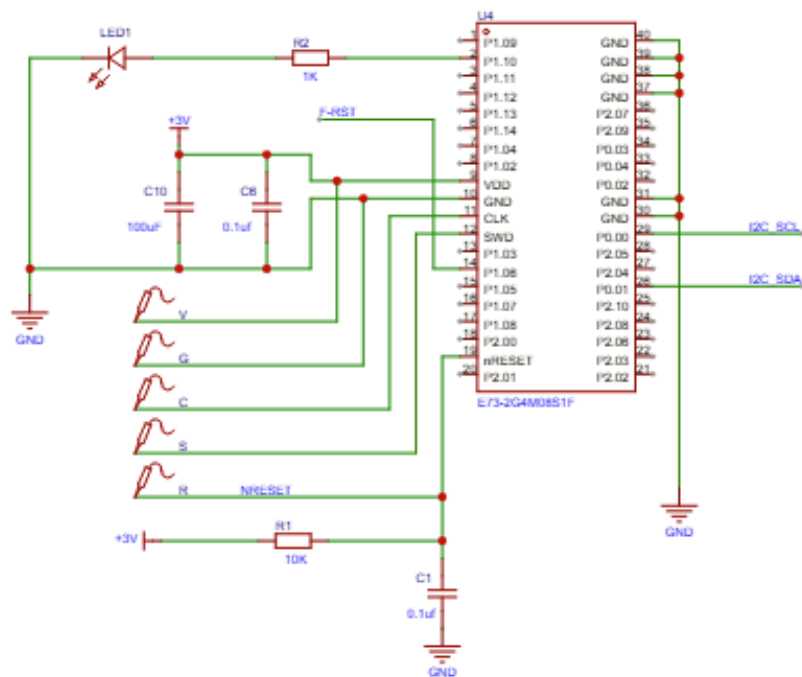
P1.10 : LED

P1.14 : Fake LED to meet Nordic Matter Board library.

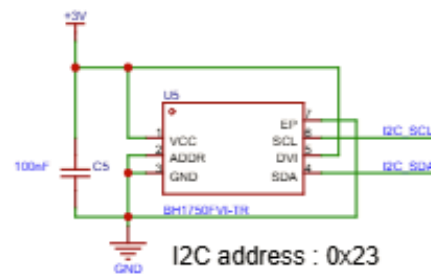
P0.00 : I2C_SCL

P0.01 : I2C_SDA

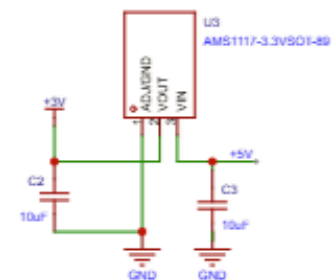
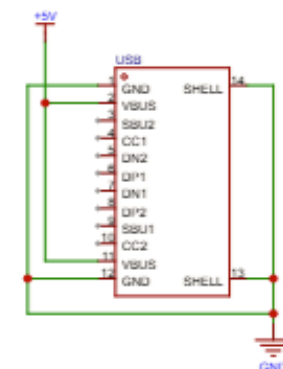
P1.06 : Factory Reset

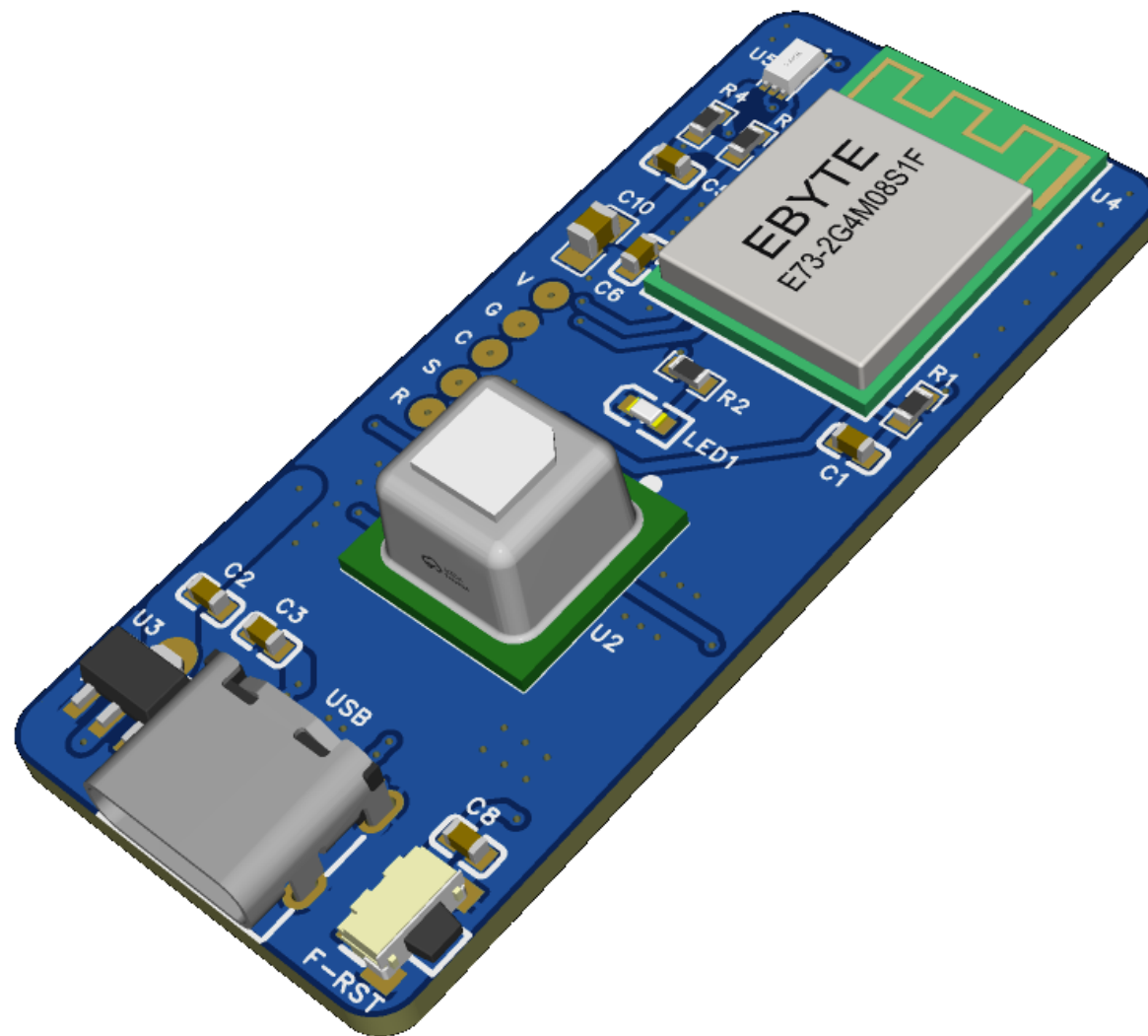


I2C address : 0x62



I2C address : 0x23





脈衝式電子門鎖 · Pulse Door Lock

apps/Pulse_Lock/

對電鎖送出一段短脈衝來開門，以標準 Matter 門鎖的形式出現在 App 裡。

產品代碼	Product ID	Matter 裝置類型	端點	供電 / Thread 角色	Bootloader / OTA	已部署
12	0x800C	0x000A Door Lock	2	市電 / Thread 路由節點	MCUboot + BLE DFU	0 台

硬體重點	Matter 資料模型	韌體模組
· 脈衝輸出 → P0.01 · 無感測器 — 目前是單向開鎖 · 不回報實體上鎖狀態 · 接腳配置與 camera_switch 相同	· EP1 DoorLock (LockState 、 Lock/Unlock) · 共 2 個端點	· pulse_switch.cpp (脈衝时序) · reboot_fix.cpp 、 watchdog.cpp

GPIO 一覽 (含板上共用的 LED 與按鍵)			
P0.01	脈衝輸出 → 電鎖 (ACTIVE_HIGH)	P1.10	狀態 LED (配網時亮)
P1.06	回復原廠按鍵 (長按 ≥ 6 秒)	P1.14	保留 LED (框架需要 · 未接線)

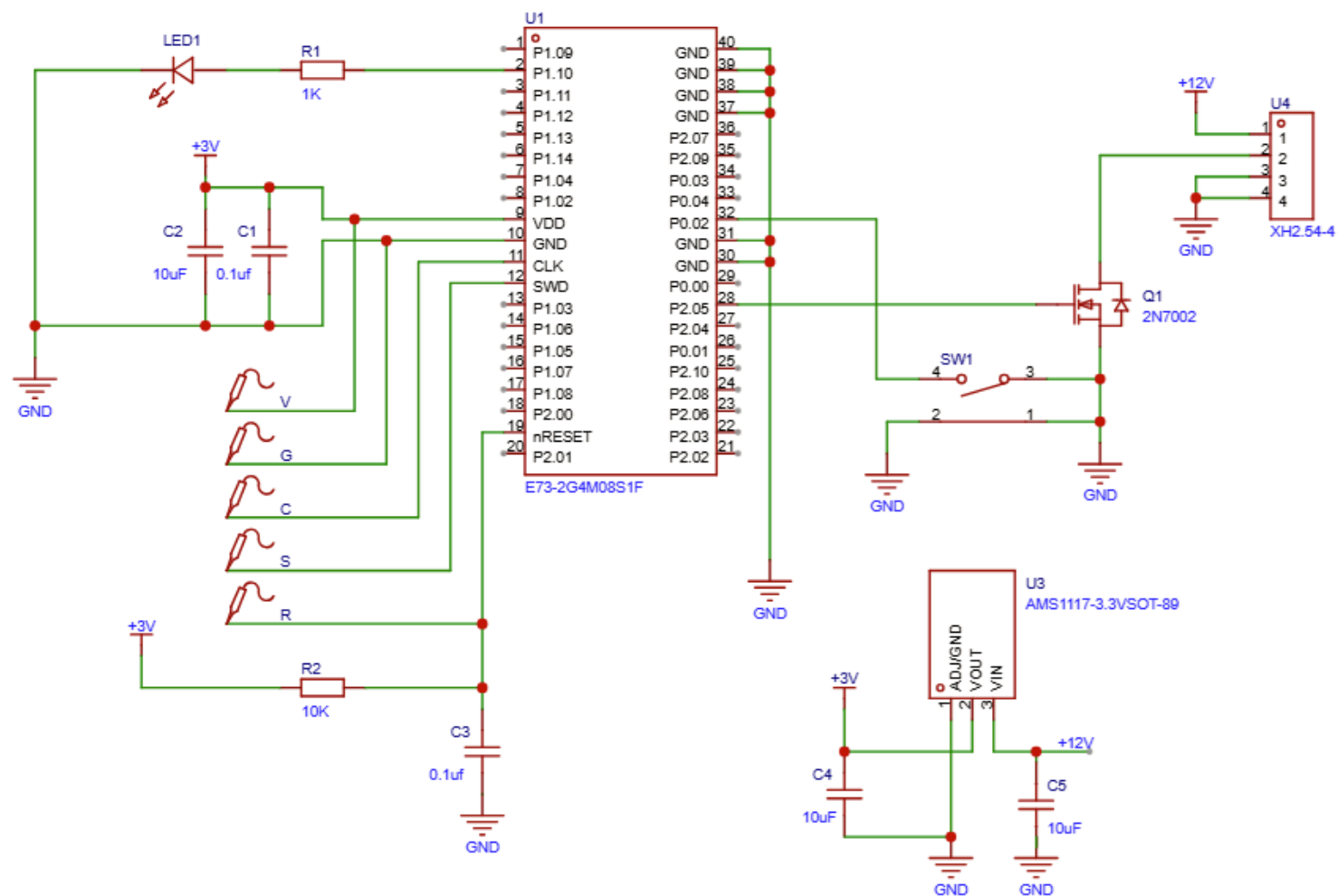
實戰筆記

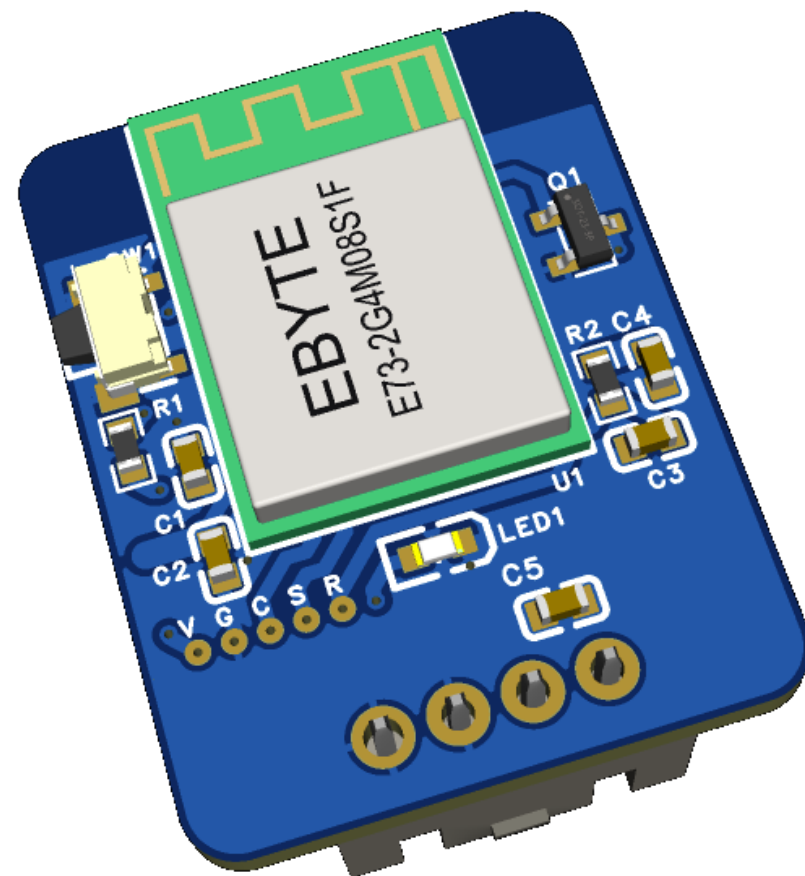
韌體已完成並通過建置，但尚未安裝到任何一台實機 (已部署 0 台) 。

脈衝式電子門鎖 · Pulse Door Lock

電路圖

GPIO
P1.10 : LED
P2.05 : HALL Sensor
P0.02 : Factory Reset





電風扇控制器 · Fan Controller

apps/fan_control/

以「模擬按下原廠按鍵」的方式控制既有電風扇，並讀取回授訊號來反推真實狀態。

產品代碼	Product ID	Matter 裝置類型	端點	供電 / Thread 角色	Bootloader / OTA	已部署
15	0x800F	0x002B Fan	2	市電 / Thread 路由節點	MCUboot + BLE DFU	2 台

硬體重點	Matter 資料模型	韌體模組
· 模擬按鍵三支都在 port 2 · 回授訊號兩支在 port 1 (可中斷) · 轉速 PWM 原本在 P1.03，為硬體驗證改到 P1.05 · 不直接驅動馬達，只是「代替人按按鍵」	· EP1 FanControl (風速、模式) · EP1 OnOff · 擺頭以 RockSetting / 自訂屬性表示	· fan_control.cpp (送出模擬按鍵) · fan_monitor.cpp (量 PWM 週期推算檔位) · dfu_advertising.cpp

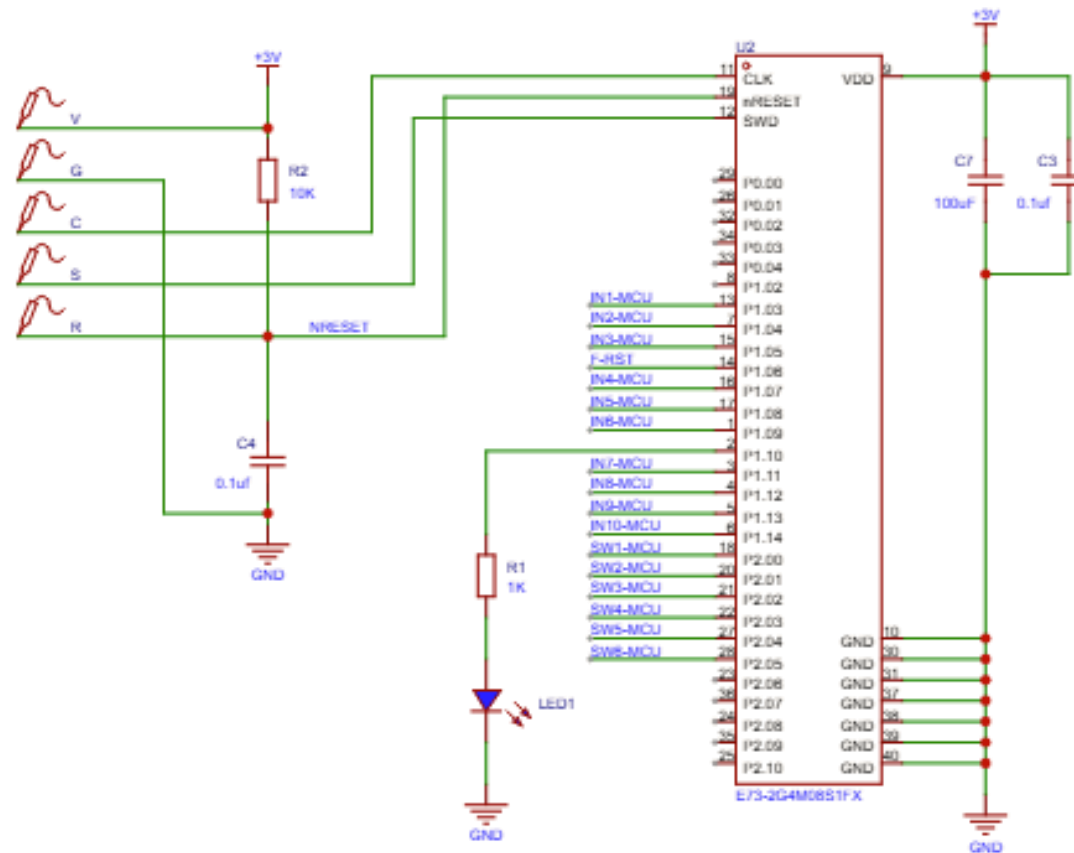
GPIO 一覽 (含板上共用的 LED 與按鍵)					
P2.00	模擬按鍵：電源開關	P1.05	回授：轉速 PWM (量週期推算檔位)	P1.06	回復原廠按鍵 (長按 ≥ 6 秒)
P2.01	模擬按鍵：擺頭開關	P1.04	回授：擺頭脈衝 (判斷有無邊緣)	P1.10	狀態 LED (配網時亮)
P2.02	模擬按鍵：風速 +1 檔	回授腳	皆設 GPIO_PULL_UP	P1.14	保留 LED (框架需要，未接線)

實戰筆記

因為是「按遙控器」而不是直接驅動，狀態一定要靠回授同步 — 否則 App 顯示會和實際不一致。

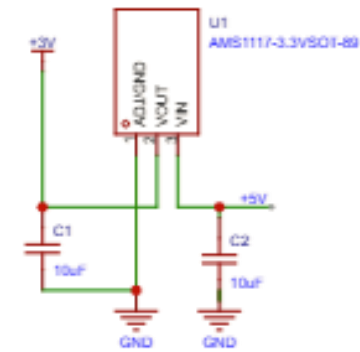
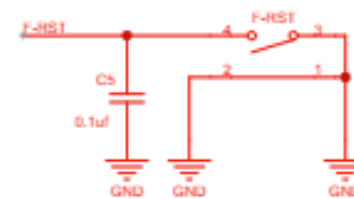
電風扇控制器 · Fan Controller

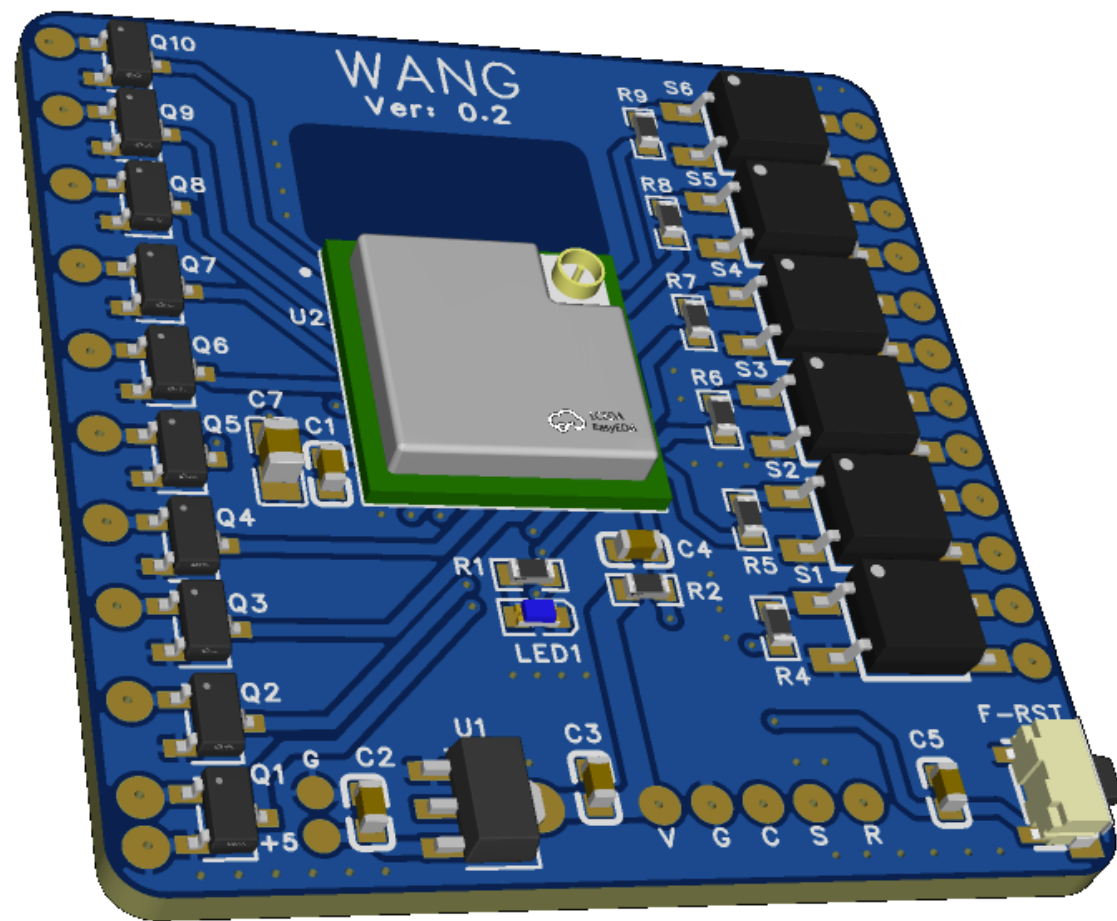
電路圖



GPIO
P1.10: LED
P1.14: Fake LED
P1.03: IN1-MCU
P1.04: IN2-MCU
P2.00: BUTTON1
P2.01: BUTTON2
P2.02: BUTTON3
P1.06: Factory Reset

to detect spin level
to detect swing on/off
for FAN on/off
for swing on/off
for spin level control, one level pre trigger





把 12 段風速壓成 4 段 — 小愛的限制

Home Assistant 與 HomeKit 用「百分比」，但巴法雲 (Bemfa) 送給小愛的風速只有 1–4 段：

```
巴法雲 fan 主題 ( 類型後綴 003 ) 的訊息格式：  
on|off # 風速(1-4) # 擺頭
```

- 我們的風扇有 12 段 ($\text{percentage_step} = 8.333\%$)，必須壓縮成 4 段
- 風速 0 在協定裡「不合法」— 語音助理會把 on#0#... 直接解讀成「關閉」
- 所以風扇開著時永遠不回報 0，最低退回第 1 段

段位 (小愛)	1	2	3	4
百分比 (HA / HK)	25%	50%	75%	100%

兩條換算路徑，依裝置自動選擇

custom_components/bemfa/sync_fan.py 會依裝置能力挑一條路：

路徑 A – 百分比

```
回報：clamp(ceil(pct / 25), 1..4)
下令：fan.set_percentage(gear * 25)
```

路徑 B – preset 模式

```
low:1  medium:2
high:3  auto:4
```

- 為什麼需要兩條？因為同型號、同韌體的兩台風扇行為並不一樣：
 - 客廳那台永遠回報 percentage = 0，即使正在用 preset 運轉 → 走路徑 B
 - 主臥那台會回報真實百分比 → 走路徑 A
- ⚠ 第 4 段對應的是 auto，不是「最強」—— high 其實在第 3 段
 - 要讓「最大」等於 high，改 PRESET TO GEAR 一行就好

HACS 更新整合套件時會「無聲覆蓋」sync_fan.py。段位行為一變怪，先檢查這個檔案；而且改完必須完整重啟 HA 才會生效。

車庫門控制器 · Garage Door Controller

apps/Door_Controller/

控制捲門馬達的開 / 停 / 關，回報門的實際位置，並同時接收原廠 433 MHz 遙控器的按鍵。

產品代碼	Product ID	Matter 裝置類型	端點	供電 / Thread 角色	Bootloader / OTA	已部署
17	0x8011	0x0202 Window Covering	3	市電 / Thread 路由節點	MCUboot + BLE DFU	1 台

硬體重點

- 雙 AS5600 游標式編碼器：細 1:1、粗 1:31
- 細的走硬體 TWIM30 (port 0 唯一的序列埠)
- 粗的走軟體 gpio-i2c，用 P1.02/03 的 NFC 腳位
- CC1101 433 MHz OOK 接收，讀 RSSI / LQI
- CT → P1.04 = AIN0 (2026-08-26 確認，不是 AIN4)
- 按鍵 P1.11–13 與門磁 P1.14 佔掉 AIN4–AIN7

Matter 資料模型

- EP1 WindowCovering (開/停/關、位置)
- EP2 BooleanState (門磁接點)
- ElectricalPowerMeasurement (馬達電流)

韌體模組

- door_control.cpp (動作狀態機)
- rf_receiver.cpp (CC1101 OOK 解碼)
- encoder.cpp、contact_sensor.cpp、metering.cpp

GPIO 一覽 (含板上共用的 LED 與按鍵)

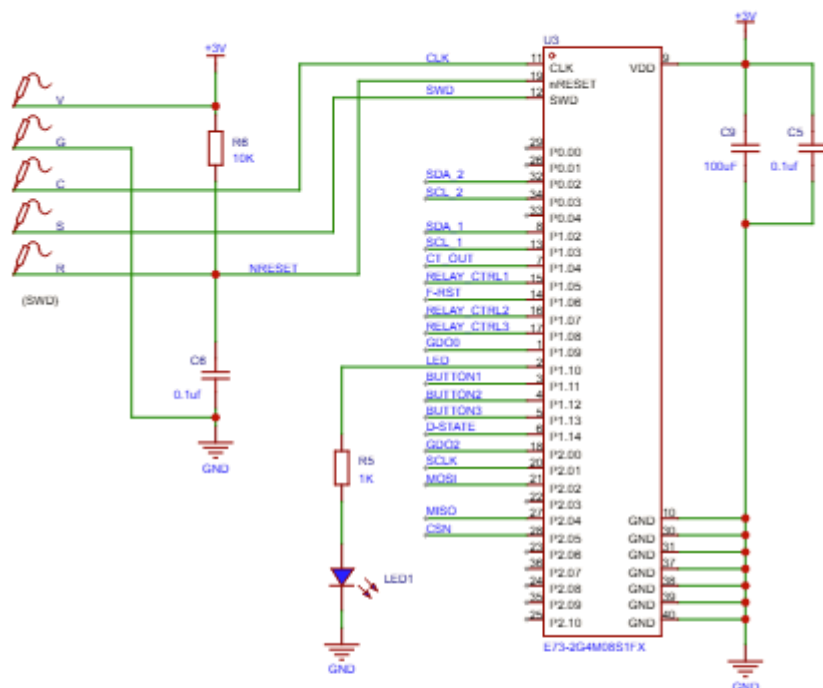
P1.05	繼電器：開門	P1.13	實體按鍵：開門	P2.02	CC1101 MOSI	P1.03	軟體 I2C SCL → AS5600 粗編碼器
P1.07	繼電器：停止	P1.14	門磁接點 (經 2N7002 N-FET)	P2.04	CC1101 MISO (= GDO1)	P1.04	電流互感器 CT → SAADC AIN0
P1.08	繼電器：開門	P1.09	CC1101 GDO0：OOK 資料	P0.02	I2C30 SDA → AS5600 細編碼器	P1.06	回復原廠按鍵 (長按 ≥ 6 秒)
P1.11	實體按鍵：關門	P2.05	CC1101 CSN	P0.03	I2C30 SCL → AS5600 細編碼器	P1.10	狀態 LED (配網時亮)
P1.12	實體按鍵：停止	P2.01	CC1101 SCK	P1.02	軟體 I2C SDA → AS5600 粗編碼器	P0.04	保留 LED — 因 P1.14 被佔用而改接

實戰筆記

整個產品線中最複雜的一個：繼電器、RF 接收、硬體 + 軟體雙 I2C、CT 量測與門磁輸入全都有，幾乎用光所有腳位。

車庫門控制器 · Garage Door Controller

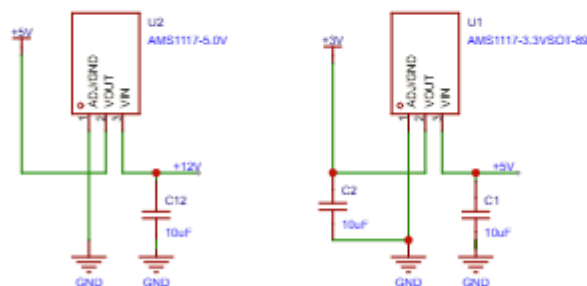
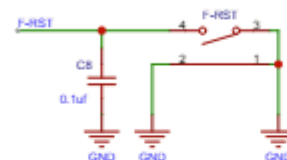
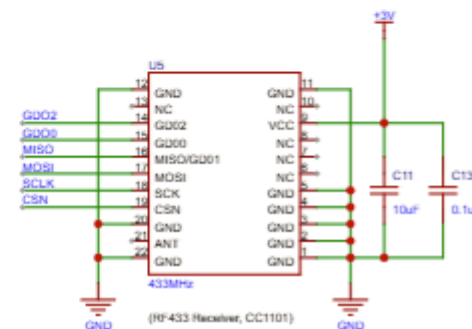
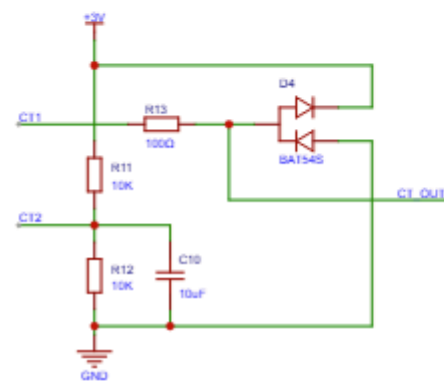
電路圖 1 / 2



GPIO

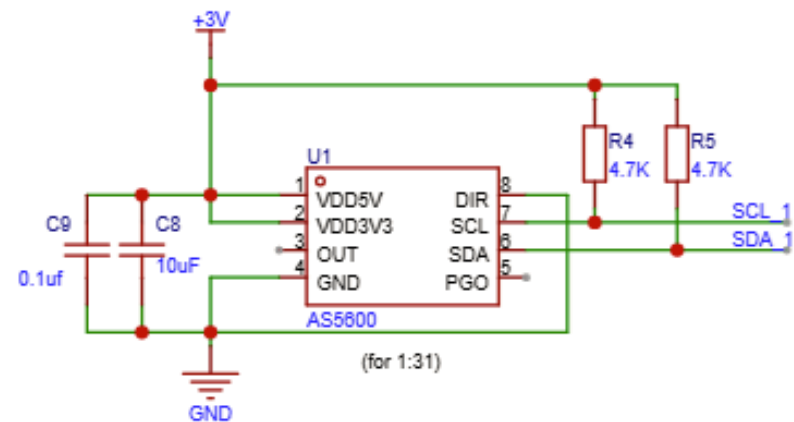
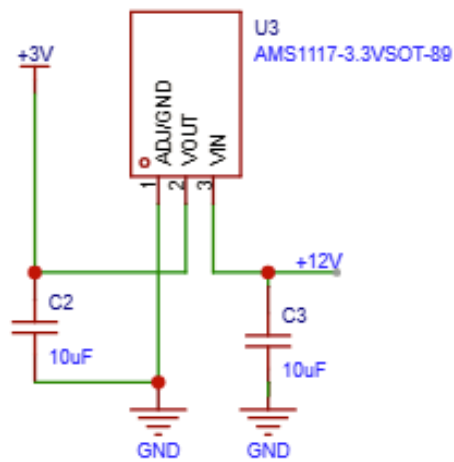
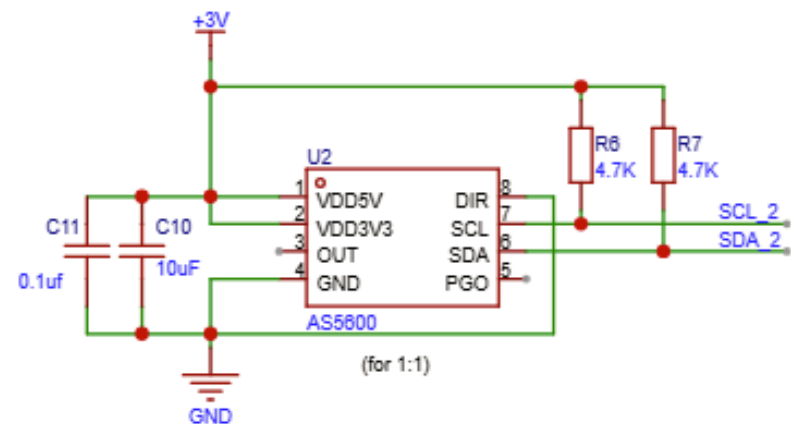
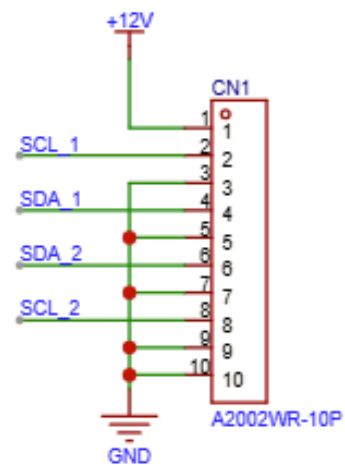
P1.04 : CT output; 20A/1V; 1.65V bias
P1.10: LED
P1.14: D-STATE (Door sensor)
P1.05: RELAY_CTRL1 (Close)
P1.07: RELAY_CTRL2 (Stop)
P1.08: RELAY_CTRL3 (Open)
P1.11: BUTTON1 (Close)
P1.12: BUTTON2 (Stop)
P1.13: BUTTON3 (Open)
P1.06: Factory Reset

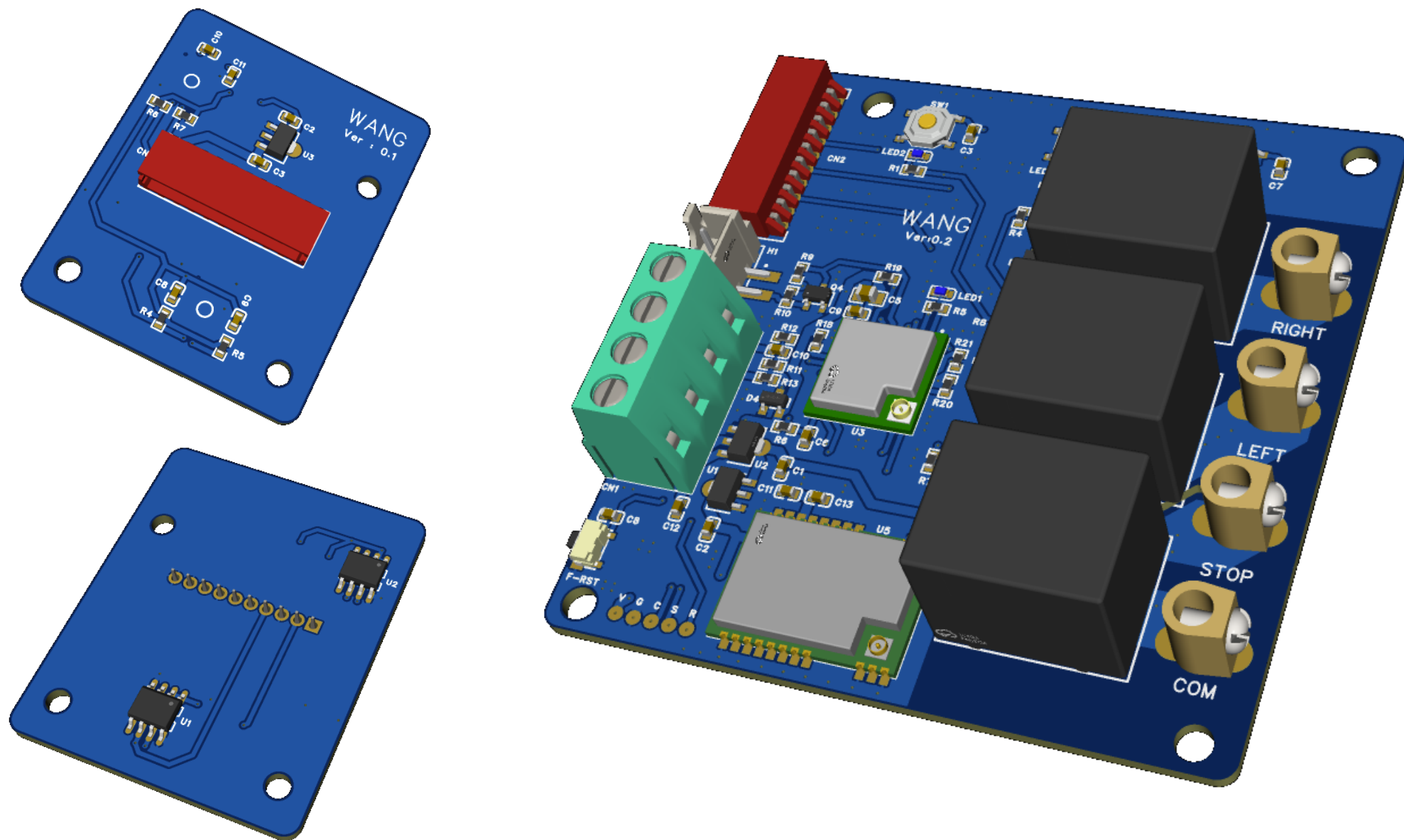
P0.02 : SDA_2 (I2C for main AS5600)
P0.03 : SCL_2 (I2C for main AS5600)
P1.02 : SDA_1 (I2C for 2nd AS5600)
P1.03 : SCL_1 (I2C for 2nd AS5600)
P2.00 : GDO2 (for CC1101)
P2.01 : SCLK (for CC1101)
P2.02 : MOSI (for CC1101)
P2.04 : MISO (for CC1101)
P2.05 : CSN (for CC1101)
P1.09 : GDO0 (for CC1101)



車庫門控制器 · Garage Door Controller

電路圖 2/ 2





溫濕度感測器（路由型） · Temp / Humidity Router

apps/temphumidity-router/

和電池版感測同樣的東西，但改用市電供電，因此能當 Thread 路由節點擴大網狀網路覆蓋。

產品代碼	Product ID	Matter 裝置類型	端點	供電 / Thread 角色	Bootloader / OTA	已部署
18	0x8012	0x0302 Temperature Sensor	3	市電 / Thread 路由節點	MCUboot + BLE DFU	1 台

硬體重點	Matter 資料模型	韌體模組
· SHT35 @ 0x44 (溫度 + 濕度) · i2c30 : SCL P0.00 / SDA P0.01 · 市電供電，無電池量測電路 · 與電池版硬體幾乎相同，只差供電	· EP1 TemperatureMeasurement · EP2 RelativeHumidityMeasurement · 無 ICD — 永遠在線，可轉發封包	· temp_humidity_sensor.cpp、ble_config_service.cpp、watchdog.cpp

GPIO 一覽 (含板上共用的 LED 與按鍵)			
P0.00	I2C30 SCL (TWIM30)	P1.06	回復原廠按鍵 (長按 ≥ 6 秒)
P0.01	I2C30 SDA (TWIM30)	P1.10	狀態 LED (配網時亮)
I2C 0x44	SHT35 溫濕度感測器	P1.14	保留 LED (框架需要，未接線)
—	市電供電，無電池量測 ADC		

實戰筆記

很好用的一招：把感測器做成市電版放在網狀網路邊緣，順便補強訊號覆蓋。

電路圖

GPIO

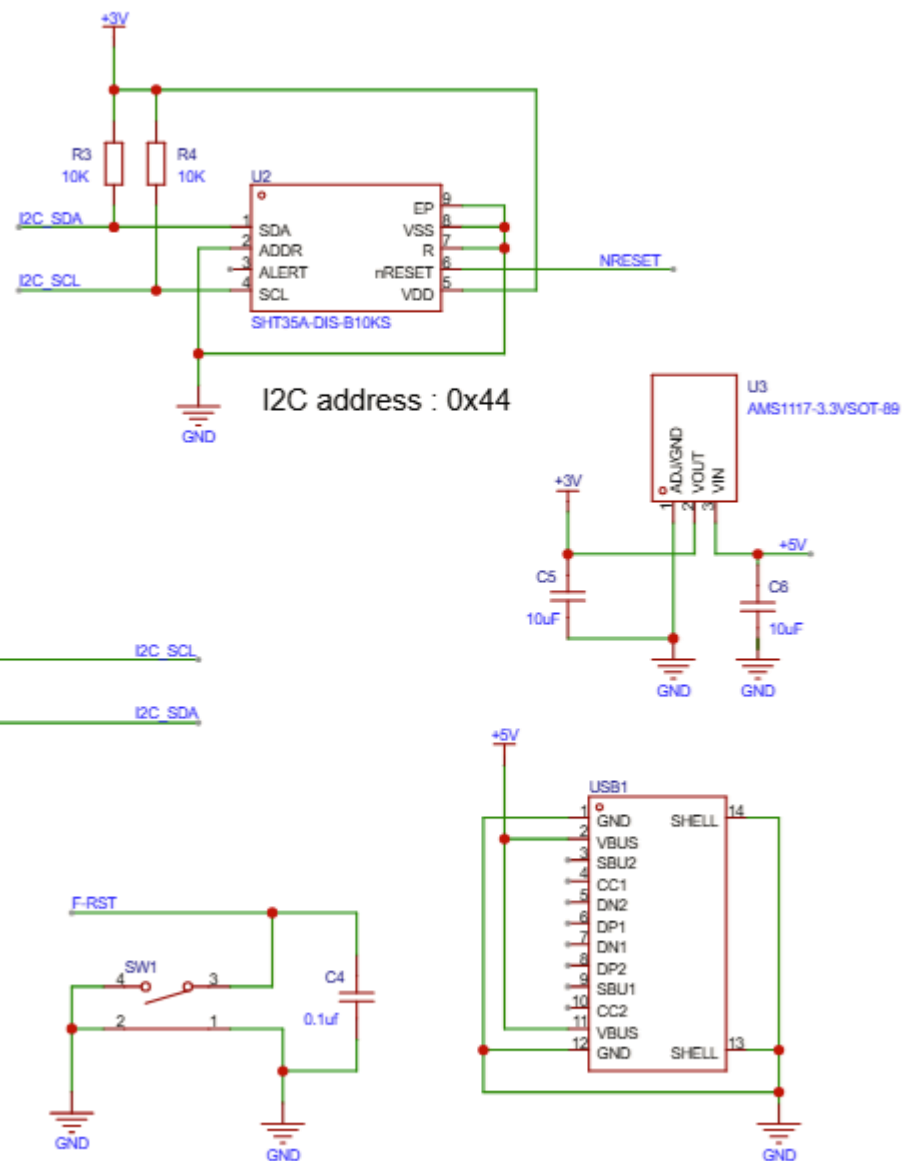
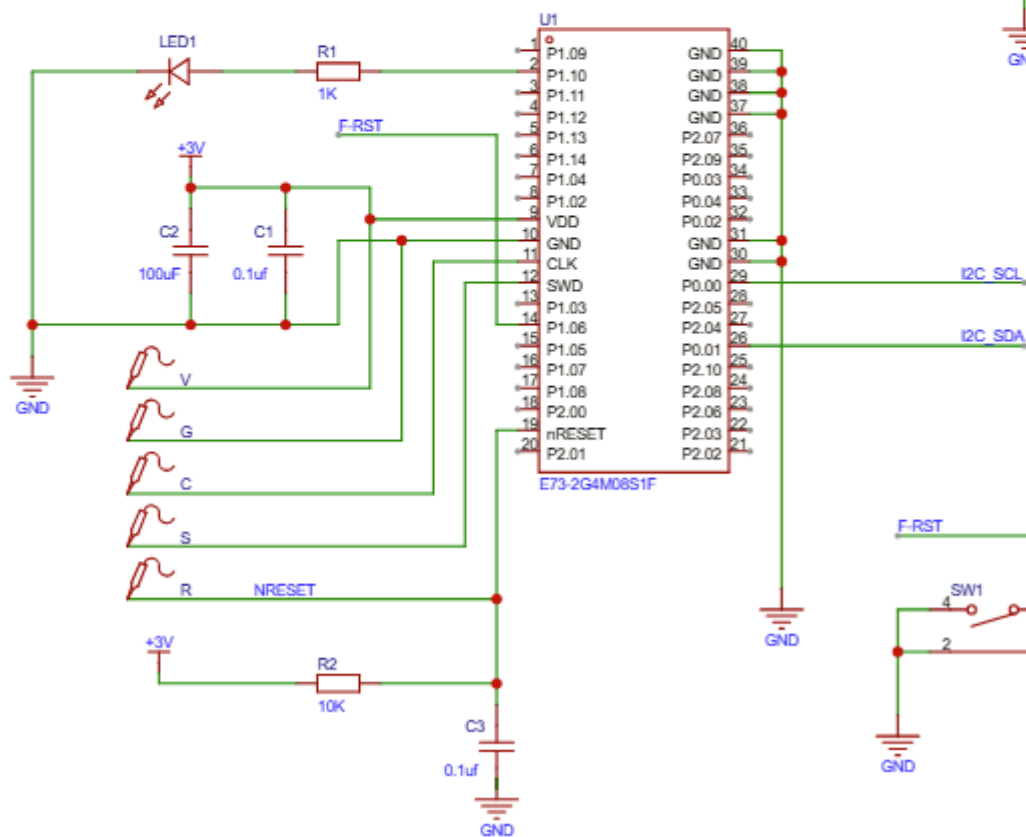
P1.10 : LED

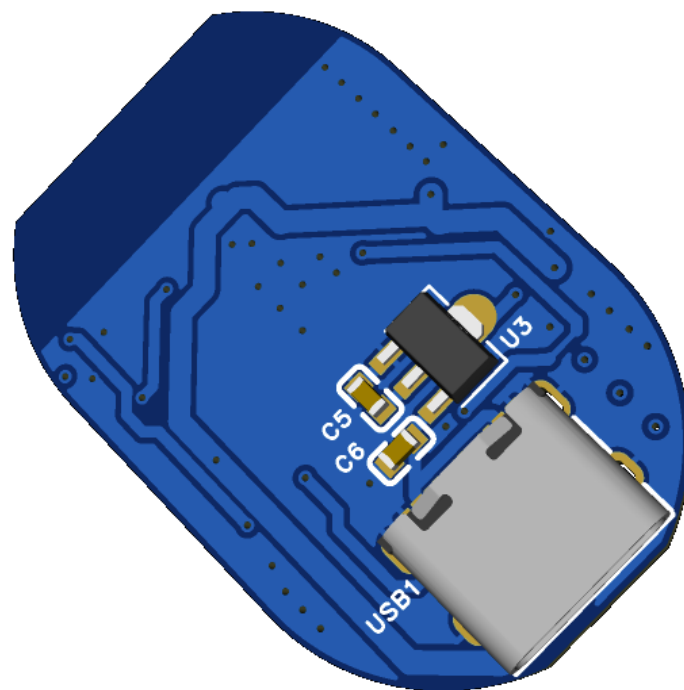
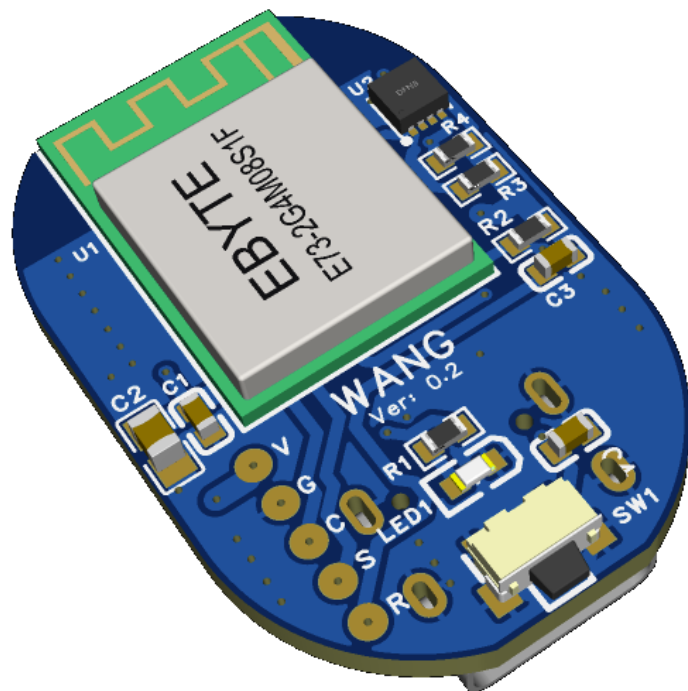
P1.14 : Fake LED to meet Nordic Matter Board library.

P0.00 : I2C SCL

P0.01 : I2C SDA

P1.06 : Factory Reset





溫濕度感測器 · Temp / Humidity Sensor

apps/TempHumidity_Sensor_SHT35/

由 BMP280 版本拆分而來：保留同一顆 SHT35，拿掉氣壓感測器與氣壓端點。

產品代碼	Product ID	Matter 裝置類型	端點	供電 / Thread 角色	Bootloader / OTA	已部署
19	0x8013	0x0302 Temperature Sensor	3	電池 / ICD 睡眠裝置	無 (v1 無 bootloader)	1 台

硬體重點	Matter 資料模型	韌體模組
· SHT35 @ 0x44 (溫度 + 濕度) · i2c30 : SCL P0.00 / SDA P0.01 · SAADC 內部 VDD 量測電池 · 與 BMP280 版唯一差別：沒有 0x76 那顆	· EP1 TemperatureMeasurement · EP2 RelativeHumidityMeasurement · EP1 PowerSource (電池)	· temp_humidity_sensor.cpp、battery_monitor.cpp、watchdog.cpp

CR2032 續航估算	可用電量 ~200 mAh	喚醒週期 300 秒	平均電流 ~5.3 µA	理論壽命 ~4.3 年	實務預期 1.5–2 年 (每 300 秒讀 SHT35 一次，約 15 ms)
----------------	------------------	---------------	-----------------	----------------	---

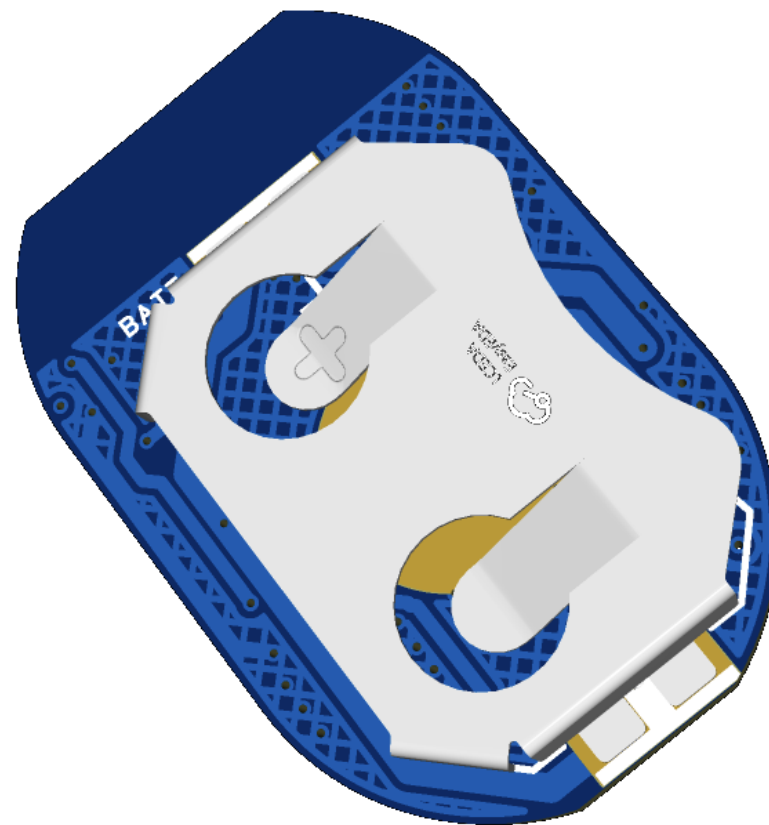
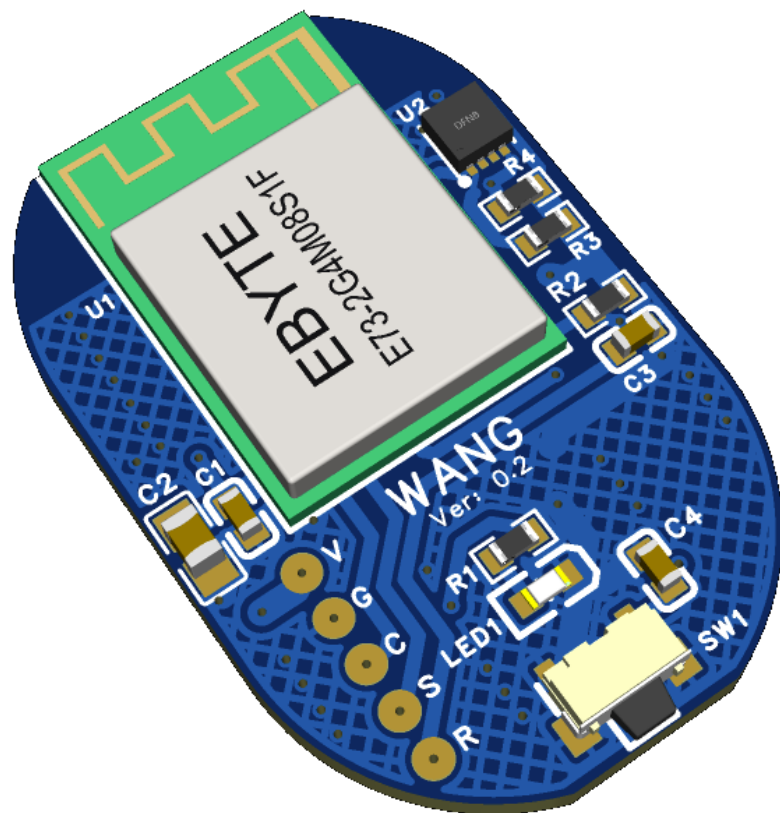
GPIO 一覽 (含板上共用的 LED 與按鍵)					
P0.00	I2C30 SCL (TWIM30)	P1.06	回復原廠按鍵 (長按 ≥ 6 秒)		
P0.01	I2C30 SDA (TWIM30)	P1.10	狀態 LED (配網時亮)		
I2C 0x44	SHT35 溫濕度感測器	P1.14	保留 LED (框架需要，未接線)		
SAADC	電池電壓 (內部 VDD，無外部分壓)				

實戰筆記

拆分產品時撞上教訓 4：新 PID 落在複製過來的舊 CD 涵蓋範圍之外，完全配不進去。

P1.06 : Factory Reset





智慧插座 (HLW8032 計量版) · Smart Plug (HLW8032 metering)

apps/AC_SmartPlug/

功能上就是一般的智慧插座。與 Smart_Plug 的唯一差別在計量方式：這一版用 HLW8032 電表 IC，可同時量電壓與電流；Smart_Plug 用 CT，只量電流。

產品代碼	Product ID	Matter 裝置類型	端點	供電 / Thread 角色	Bootloader / OTA	已部署
20	0x8014	0x010A On/Off Plug-in Unit	2	市電 / Thread 路由節點	MCUboot + BLE DFU	1 台

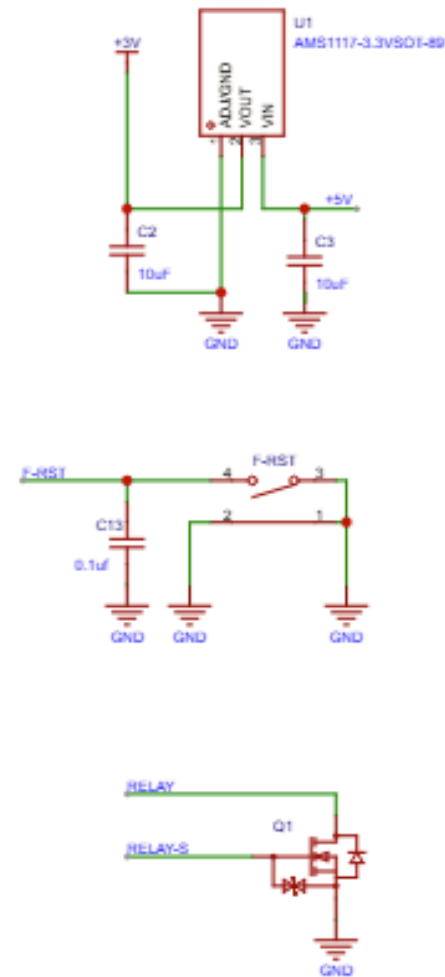
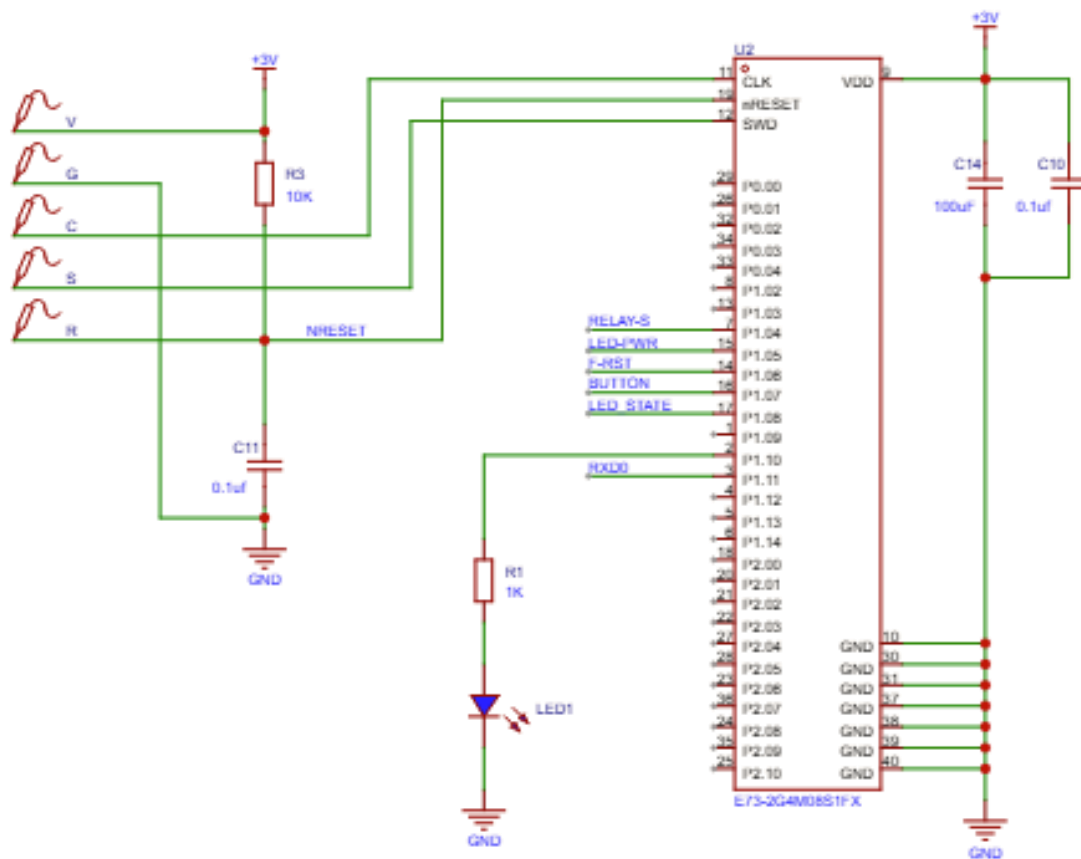
硬體重點	Matter 資料模型	韌體模組
- 計量用 HLW8032 電表 IC：可量電壓 + 電流 - 本產品完全沒有 ADC — &adc 節點是刻意不放的 - HLW8032 自己連續送資料，每 50 ms 一個 24 byte 封包 ⚠ 與 Smart_Plug 相比 P1.04 / P1.05 角色對調 - 兩顆按鍵都由 App 接管，共用同一個處理常式	- EP1 OnOff (狀態持久化，重開機後恢復) - EP1 ElectricalPowerMeasurement (功率 / 電流) - EP1 Identify	- relay_control.cpp、metering.cpp - ElectricalPowerMeasurementDelegate.cpp - ble_config_service.cpp、watchdog.cpp

GPIO 一覽 (含板上共用的 LED 與按鍵)					
P1.04	繼電器 (自保持，ACTIVE_HIGH)	P1.07	使用者按鍵 (PULL_UP / ACTIVE_LOW)	UART	4800 baud、8 資料位元、偶同位、1 停止位元
P1.05	狀態 LED：跟隨繼電器	P1.06	模組上開發按鍵 (與 sw0 同腳，由 App 接管)	P1.10	狀態 LED (配網時亮)
P1.08	電源 LED：開機後恆亮	P1.11	UART20 RX ← HLW8032 (經 EL357 光耦)	P1.14	保留 LED (框架需要，未接線)

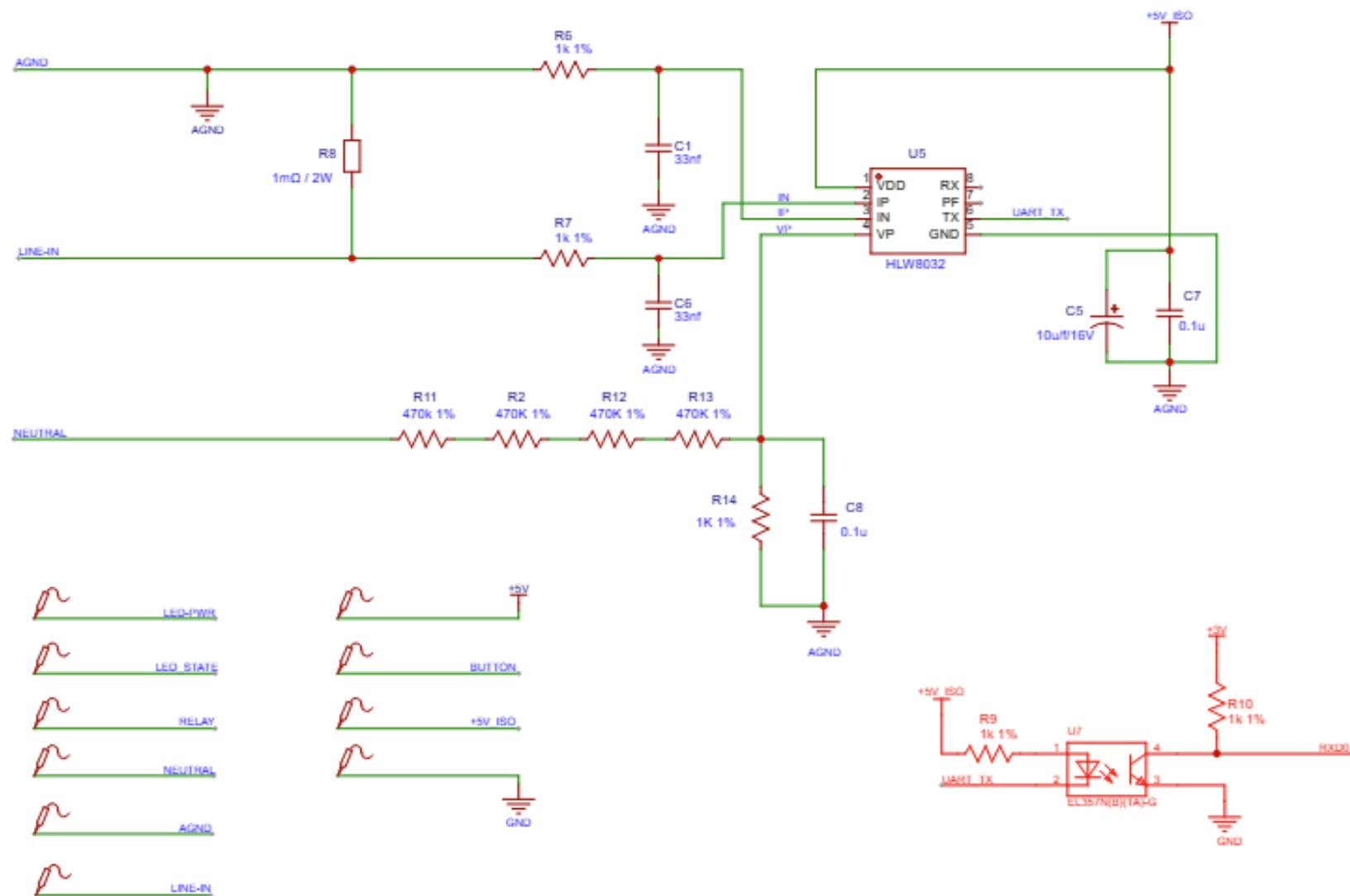
實戰筆記

2026-09-06 因佈局修正把電源 LED 與狀態 LED 對調 (P1.05 ↔ P1.08)。DT 屬性是以「功能」命名而非腳位，所以 C++ 完全不用改 — 但更早做的板子接法是相反的。

GPIO
P1.10: LED
P1.14: Fake LED
P1.05: LED-PWR
P1.04: RELAY-S
P1.11: RXD0
P1.08: LED-STATE
P1.07: BUTTON
P1.06: Factory Reset

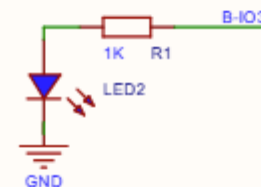
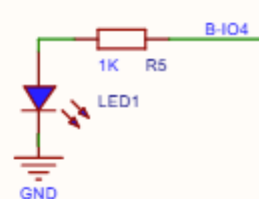
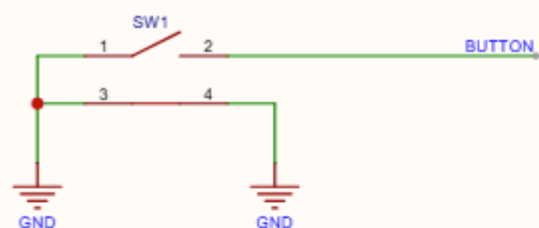
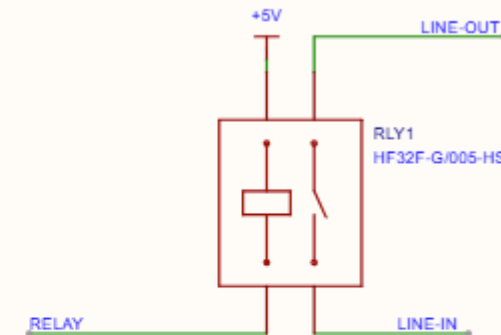
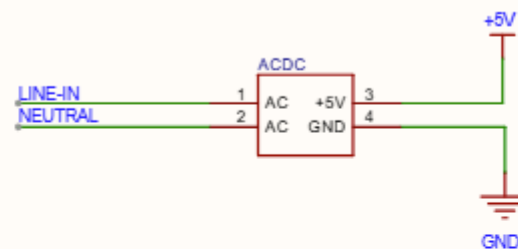
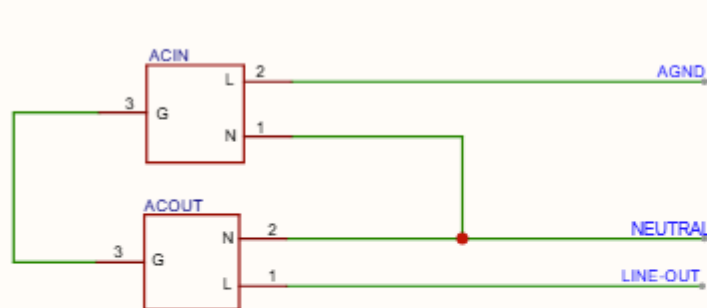


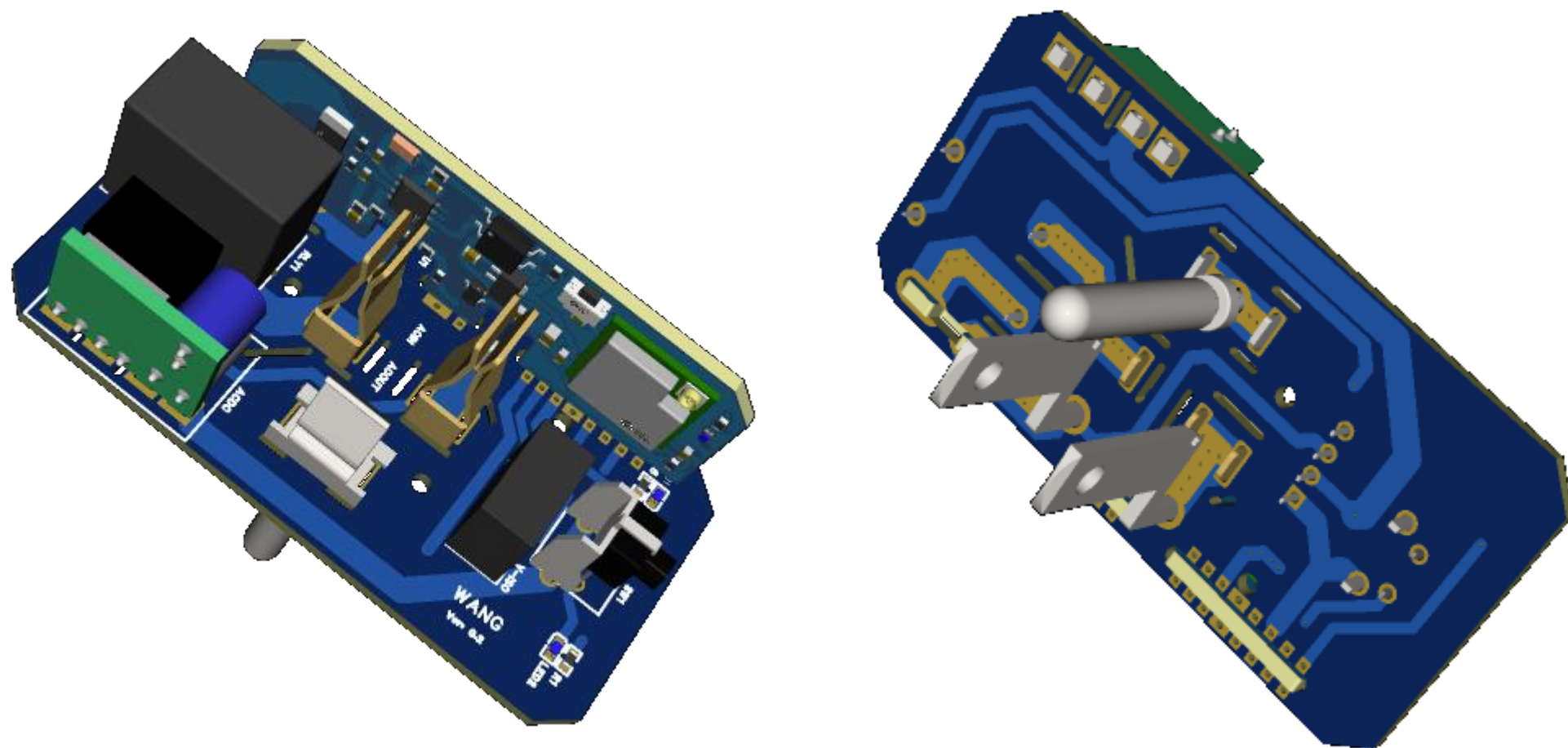
電路圖 2 / 3



智慧插座 (HLW8032 計量版) · Smart Plug (HLW8032 metering)

電路圖 3 / 3





第 10 部

經驗教訓

每一條都花了我們好幾天。

這是整份簡報最有價值的一段。

教訓 1

一個看起來像「裝置死掉」的堆疊溢位

症狀 配對時卡在「連線中」，裝置完全沒反應，要重試很多次才配得上。

根本原因 藍牙 RX 執行緒在配網過程中把 1200 bytes 的堆疊用爆了。在 ARMv8-M 上會觸發 UsageFault，Zephyr 接著讓系統「停住」——而 Debug 版並不會自動重開。

解法 把 CONFIG_BT_RX_STACK_SIZE 調到 2048。找出來的方法是用 SWD 把 CPU 停住，讀出 PSP 等於 PSPLIM。

帶走這句 當裝置「死掉」時，把核心停住、把暫存器讀出來。不要用猜的。

斷電後要六個小時才會回來

症狀 重新上電之後，感測器好幾個小時完全沒資料，然後又自己好了。重開邊界路由器也能修好它。

根本原因 Matter 的連線工作階段存在 RAM 裡，一斷電就消失。開機後裝置會用費氏數列退避演算法重試訂閱恢復——而那個退避時間的上限是「六小時」。

解法 在 `chip_project_config.h` 裡把訂閱恢復的重試上限改成 10 分鐘。

教訓 3

OTA 顯示成功，但什麼都沒變

症狀

韌體透過 BLE DFU 上傳完全沒有報錯，但裝置回報的版本還是舊的。

根本原因

我們幫專案加了映像檔簽章金鑰。但裝置上「已經存在」的 bootloader 只信任它自己被編譯進去的那把金鑰，所以開機時默默拒絕了新映像檔。

解法

用 SWD 把新的 bootloader 燒一次進去。之後 OTA 就正常了。

帶走這句

更換簽章金鑰會讓所有「已部署」裝置的 OTA 失效。要換，就在板子還在你手上時換。

教訓 4

一個全新的產品完全配不進去

症狀 新做的感測器一配網就失敗，跳出憑證驗證錯誤。PID 相鄰的舊產品卻都配得好好的。

根本原因 Certification Declaration 裡列著它涵蓋哪些 PID。而這個新產品的資料夾裡帶著一份「舊的」CD，涵蓋範圍到它的 PID 之前就停了。

解法 把最新的 CD 同步到每一個產品，並加上一致性檢查。

帶走這句 被複製出去的檔案一定會走鐘。只要有檔案是「每個產品各一份」，就必須有東西去驗證它們仍然一致。

教訓 5

兩個產品在 Home Assistant 裡變成同一台裝置

症狀 Home Assistant 顯示出「一台」裝置，底下卻掛著兩個完全不同產品的實體。

根本原因 兩者因為複製貼上，帶著同一組寫死的序號。而 HA 的裝置註冊表「只用序號」當主鍵。

解法 改成從 SoC 的唯一 ID 在執行期推導序號，衝突就變成不可能發生。

帶走這句 任何「用手打出來」的身分識別，遲早都會重複。改成用推導的。

電源管理默默把感測器關掉了

症狀 氣壓感測器回傳一個凍住、而且不可能的數值 — 但「同一條 I2C 匯流排」上的另一顆感測器完全正常。

根本原因 執行期電源管理會把周邊掛起。驅動的 suspend 處理常式把感測器切進睡眠模式，它就停止轉換了 — 但它「還是會回應 I2C」，所以表面上看不出任何異常。

解法 針對該產品關閉 CONFIG_PM_DEVICE_RUNTIME，或逐一確認每個驅動的 PM 行為。

帶走這句 「會回應的感測器」不等於「有在量測的感測器」。

山寨與不良的感測器

症狀 氣壓讀到約 640 hPa，而不是正常的約 1010。換了三片板子、重焊兩顆料，行為完全一樣。

根本原因 那些料件回報了正確的晶片 ID、也帶著真實的校正資料 — 但它們的「溫度通道」永遠回傳 0。而氣壓補償運算需要溫度，所以輸出就是錯的。

解法 把原始暫存器 dump 出來。直接讀校正資料與 ADC 原始值確認之後，改用比較新、比較少被仿冒的料。

帶走這句 停產 (EOL) 的料件最容易出現山寨品。怪自己的板子之前，先去讀原始暫存器。

教訓 8

沒印出來的 log 不是證據

症狀

某一行 log 一直沒出現，我們就下結論說那段程式沒被執行。發生了兩次，兩次都錯。

根本原因

Deferred (延遲) 模式的 log 在緩衝區滿的時候會「丟訊息」——而 Matter 開機時的大量 log 正好就會塞爆它。

解法

把吵雜的子系統調安靜，讓你真正在意的那幾行活下來，而且要用第二種獨立的方法去驗證。

帶走這句

「沒有 log」是「缺乏證據」，不是「證據顯示沒發生」。

Reset 不等於斷電重開

症狀 有顆感測器每次重新燒錄後就是初始化不起來，卻莫名其妙成功過一次。

根本原因 除錯器的 reset 和重新燒錄只會重啟 MCU，「不會」重啟板子上的周邊。有些料件是在自己上電的那一刻，才鎖定要走哪一種介面模式。

解法 只要是測感測器相關的東西，就把電「實際拔掉」再接回去。

帶走這句 「我 reset 過了」和「我斷電重開過了」，是兩個完全不同的測試。

教訓 10

邊界路由器會留著過期狀態

症狀 裝置配網配到一半卡住，或重新配對後就沒聲音了。但可以證明裝置本身健康而且處於閒置狀態。

根本原因 Thread 邊界路由器留著那些「已被抹除並重新配網」裝置的舊條目，而且它不會自己去重建。

解法 重開邊界路由器，並給網狀網路幾分鐘穩定下來，再去判斷是不是裝置的問題。

帶走這句 有一半的「裝置問題」其實住在控制中心裡。先證明你的裝置是閒置的。

教訓 11

複製貼上是最大的 bug 來源

症狀	某個產品的建置腳本，實際上重新產生的是「另一個產品」的檔案，而且回報成功。我們的修改被默默丟掉了好幾個月。
根本原因	那個腳本是從隔壁專案複製過來的，裡面每一條路徑都還指著原本那個專案。
解法	所有跟產品有關的值，都從資料夾名稱推導出來；絕不在複製出來的腳本裡寫死。

帶走這句

每一個你在產品之間複製的檔案，都是一顆未來的地雷。用「產生」的，不要用「複製」的。

電池裝置是完全不同的物種

症狀 在市電裝置上運作良好的逾時、重試與恢復行為，換到電池裝置上就是會失敗。

根本原因 睡眠裝置只有在輪詢的那一刻才聽得到封包 — 而間隔可能長達數分鐘。任何「假設對方會很快回應」的交握流程，就是會逾時。

解法 所有跟網路有關的行為，都要在電池硬體上「另外測一次」。絕不要假設市電上的結果可以直接套用。

帶走這句 「插座上跑起來沒問題」這句話，對電池感測器完全沒有參考價值。

教訓 13

抄來的換算公式，把最大風速變成 33%

症狀 第 4 段（最大）叫下去，風扇只跑到三分之一的速度。

根本原因 整合套件原本的寫法是 $\text{gear} \times \text{percentage_step}$ 。對一台 12 段、每段 8.333% 的風扇來說， $4 \times 8.333\% = 33\%$ 。這個公式只有在「風扇剛好也是 4 段」時才正確。

解法 改成依比例換算： $\text{gear} \times 25\%$ ，四段就落在 25 / 50 / 75 / 100%。

帶走這句

別人的預設值是為「別人的硬體」寫的。換算公式一定要對照自己的段數重新推一次。

第 11 部

除錯工具箱

怎麼知道「實際上」到底發生了什麼事。

J-Link — 整個專案最重要的一把工具

- J-Link 是 SEGGER 的除錯探針，透過 SWD 兩條線接到晶片
- 它同時是四種工具，這也是為什麼它不可取代：
 - 燒錄器 — 把韌體寫進 RRAM
 - 除錯器 — 下中斷點、停住 CPU、讀寫暫存器與記憶體
 - RTT 通道 — 不占用任何 UART 腳位就能輸出 log
 - 救援工具 — 晶片鎖死或程式跑飛時，唯一還能進去的路
- 只要 4 條線：SWDIO、SWCLK、GND、VTref (VTref 用來偵測目標電壓)

只有 J-Link 能在「裝置完全沒反應」時把真相挖出來 — 教訓 1 的堆疊溢位就是靠它停住 CPU、讀出暫存器才找到的。

J-Link 實務：常用指令與常見狀況

```
nrftutil device list          # 確認探針有被認到
nrftutil device program --firmware x.hex --x-family nrf54l
nrftutil device reset --x-family nrf54l
JLinkRTTLogger -Device nRF54L15_M33 -If SWD -Speed 4000 ...
```

症狀	實際原因與處理
debug port unavailable	裝置睡著了 — 實際斷電重開再試
No devices with requested S/N	探針沒列舉出來 — 拔掉 USB 重插
reset 之後板子沒重開	RTT logger 還占著 SWD — 先關掉它
斷電後 RTT 不會自己接回來	JLinkRTTLogger 不會重連 — 手動重跑

這四種狀況我們幾乎每次燒錄都會遇到其中一種。先照這張表處理，不要一看到錯誤就懷疑是韌體壞了。

RTT Log

- SEGGER RTT 透過 SWD 除錯器輸出 log — 完全不需要 UART 腳位
- Deferred (延遲) 模式很快，但負載高的時候「會丟訊息」
- Immediate (即時) 模式不會丟，但格式化是在呼叫端的堆疊上做的
 - 它自己就可能造成堆疊溢位 — 我們就中過
- 把吵雜的子系統調安靜，讓你真正在意的那幾行活下來

```
CONFIG_USE_SEGGER_RTT=y  
CONFIG_LOG_BACKEND_RTT=y  
CONFIG_LOG_MODE_DEFERRED=y
```

除錯

把 CPU 停住，直接讀出來

當裝置沒反應、而 log 又什麼都告訴不了你的時候：

```
JLink.exe -CommandFile halt.jlink
  si SWD / speed 4000 / device nRF54L15_M33 / connect / halt / regs
mem32 0xE000ED28 4      # CFSR HFSR DFSR MMFAR
mem32 0xE000ED24 1      # SHCSR
```

讀這個

它會告訴你

XPSR 裡的 IPSR

目前是哪一種例外 (006 = UsageFault)

PSP == PSPLIM

堆疊溢位

SHCSR 的 *ACT 位元

目前有一個例外正在作用中

用 addr2line 反查 PC

精確停在哪一行

掃描匯流排，把暫存器 dump 出來

- 感測器「不會動」的時候，第一步是先證明「它到底有沒有在回應」
- 掃過 0x08 ~ 0x77 的 I2C 位址，一次開機就能知道它在不在、在哪個位址
 - 要用「讀 1 byte」— nRF 的 TWIM 做不到零長度傳輸
 - 用零長度探測會給你「假的沒有」，我們為此浪費了好幾個小時
- 接著把裝置自己的暫存器 dump 出來：ID、設定、校正資料、ADC 原始值

讀原始暫存器讓一場兩天的調查在「一次開機」內結束。我們應該一開始就做這件事，而不是留到最後。

第 12 部

開始做你自己的產品

一份可以避開上面所有坑的檢查清單。

新產品檢查清單

- 複製一個最接近的既有產品 — 然後把「每一條」複製來的路徑與常數都改掉
- 配一個新的產品代碼；PID、DAC 前綴、Discriminator 全部從它推導
- 簽發該產品專屬的 DAC，並掛在你的控制中心「信任的」PAA 底下
- 確認 CD 的涵蓋範圍包含你的新 PID
- 連結共用的身分模組，讓序號是「推導出來」而不是「打出來」的
- 在 devicetree overlay 裡描述硬體，「同時」在 Kconfig 裡啟用它的驅動
- 每次改完資料模型，都要重新產生一次 ZAP 輸出
- 先建 Debug 版（開著 log）跑通，再去建出貨版
- 如果是電池產品，一定要在電池硬體上「另外」完整測一次
- 在相信任何感測器結果之前，先「斷電重開」，不要只按 reset

去動手做點什麼吧。

這份簡報裡的每一件事，都來自今天仍在兩個家裡運轉的真實裝置。

那些「失敗」比「成功」更有用 — 希望它們能幫你省下我們花掉的時間。